

Submission of project

Title: Securing the Digital Marketplace: Strengthening Security for Vulnerable Websites

Submitted by: ASHISH YADAV

Date: 09/11/2025

Objective

To conduct a security review, identify high-risk vulnerabilities (e.g., SQL Injection, XSS), evaluate key features for weaknesses, simulate attacks, and implement security measures to enhance website protection and integrity.

Solution — implementation overview

The project was executed in four key phases — **Reconnaissance**, **Vulnerability Exploitation**, **Access Control Testing**, and **Reporting**.

1. Reconnaissance:

The process began by performing initial reconnaissance using **Burp Suite** on the OWASP Juice Shop application. This phase helped identify publicly accessible pages and endpoints that revealed sensitive data. By intercepting and modifying HTTP requests, hidden files such as `/ftp/legal.md` and `/ftp/acquisitions.md` were accessed, confirming the presence of a **Sensitive Data Exposure** vulnerability.

2. Vulnerability Exploitation:

In this phase, various types of attacks were simulated to test the security posture of the web application.

- A **Cross-Site Scripting (XSS)** vulnerability was confirmed by injecting a malicious payload (`<img src=x onerror=prompt(document.cookie)>`) into the search field, which triggered a popup.
- A **SQL Injection (SQLi)** vulnerability was detected on the login page by using the payload `' or 1=1 --`, allowing admin-level access without valid credentials.
- **Poor Error Handling** was also identified when modifying API endpoints led to internal server errors, revealing implementation details.
- Additionally, a **Privacy Policy Exposure** issue was found, where sensitive content was accessible without proper authentication.

3. Access Control Testing:

Further testing revealed **Broken Access Control** vulnerabilities. By modifying the source code and accessing hidden paths (e.g., `/#/administration`), the administrative panel became accessible without login authentication.

Database-level vulnerabilities were also tested through SQLi-based payloads, confirming **Database Exfiltration** issues. Moreover, the discovery of a **Misplaced SIEM Signature File** exposed sensitive server-side monitoring data, posing a significant security risk.

4. Reporting and Recommendations:

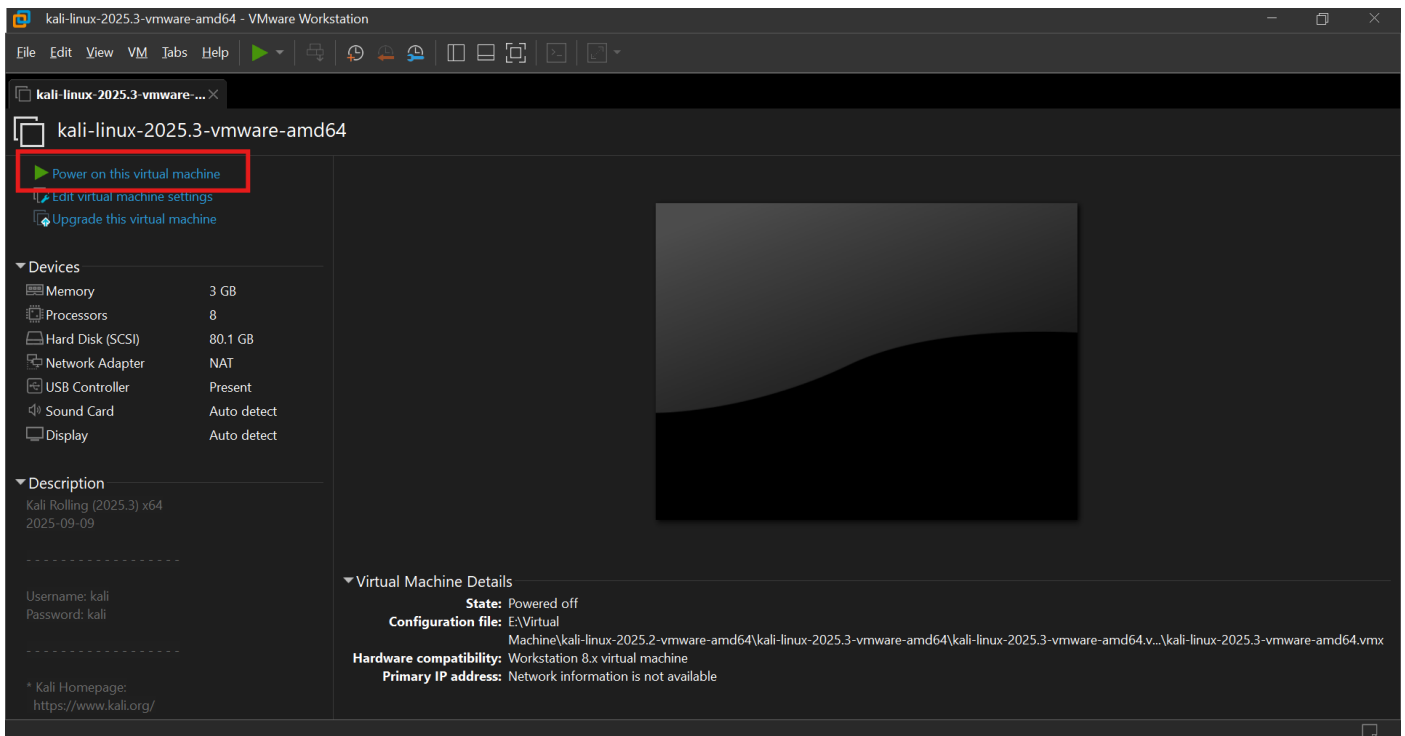
After identifying each vulnerability, detailed findings and mitigation steps were documented. The key recommendations included:

- Implementing strict **input validation and sanitization** to prevent XSS and SQLi attacks.
- Applying **role-based access control (RBAC)** to restrict unauthorized users.
- Configuring **error handling** to avoid exposing server details.
- Deploying **Web Application Firewall (WAF)** and **Security Information and Event Management (SIEM)** tools for proactive threat detection.

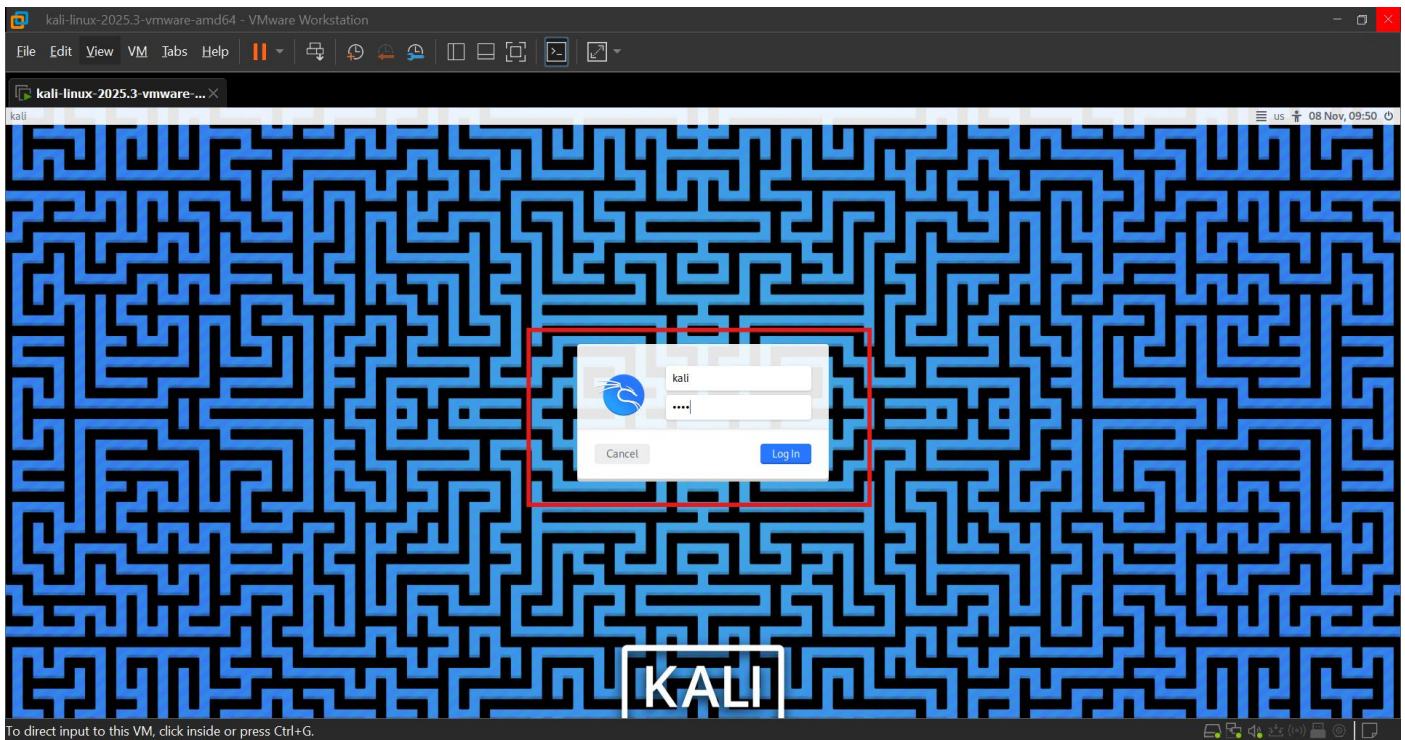
- Conducting regular **security audits** and following secure development practices.

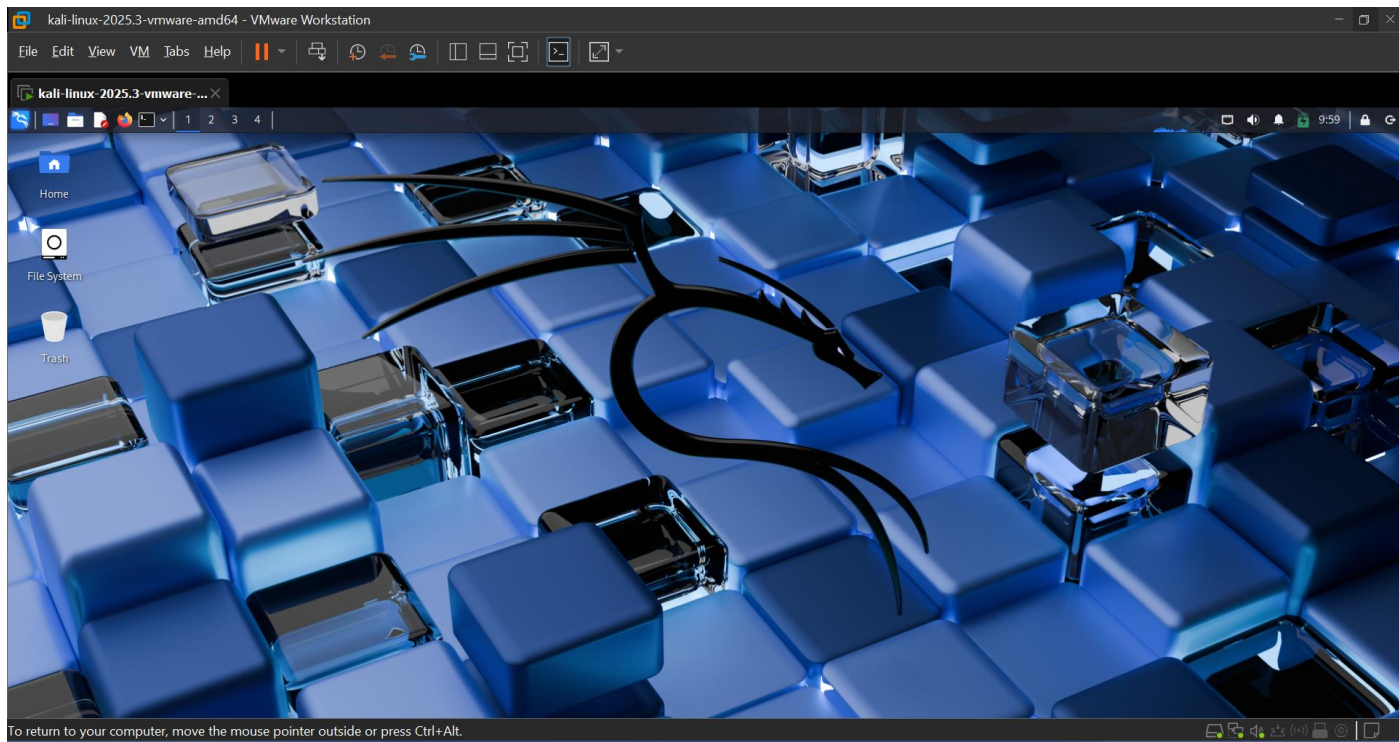
Task 1: Perform Reconnaissance

1.1 Powering on the kali virtual machine

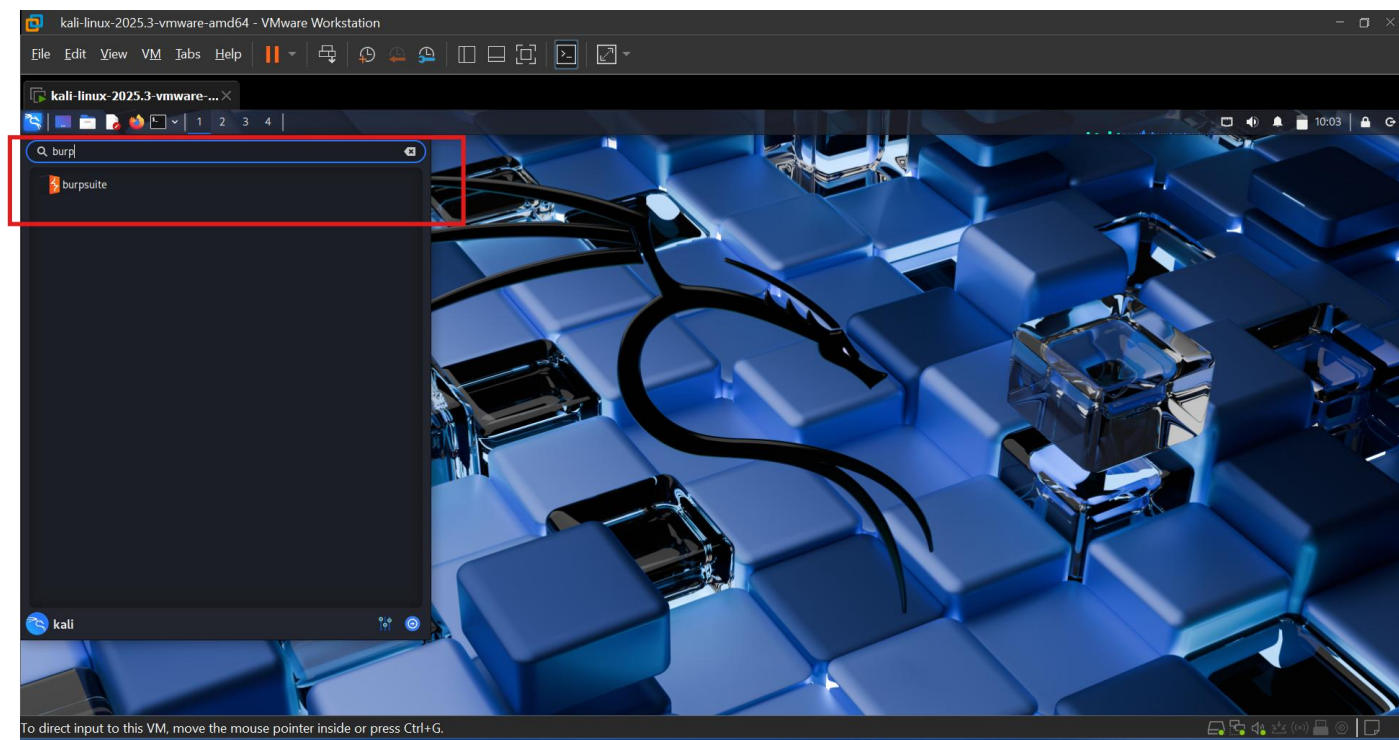


1.2 Logging using credentials

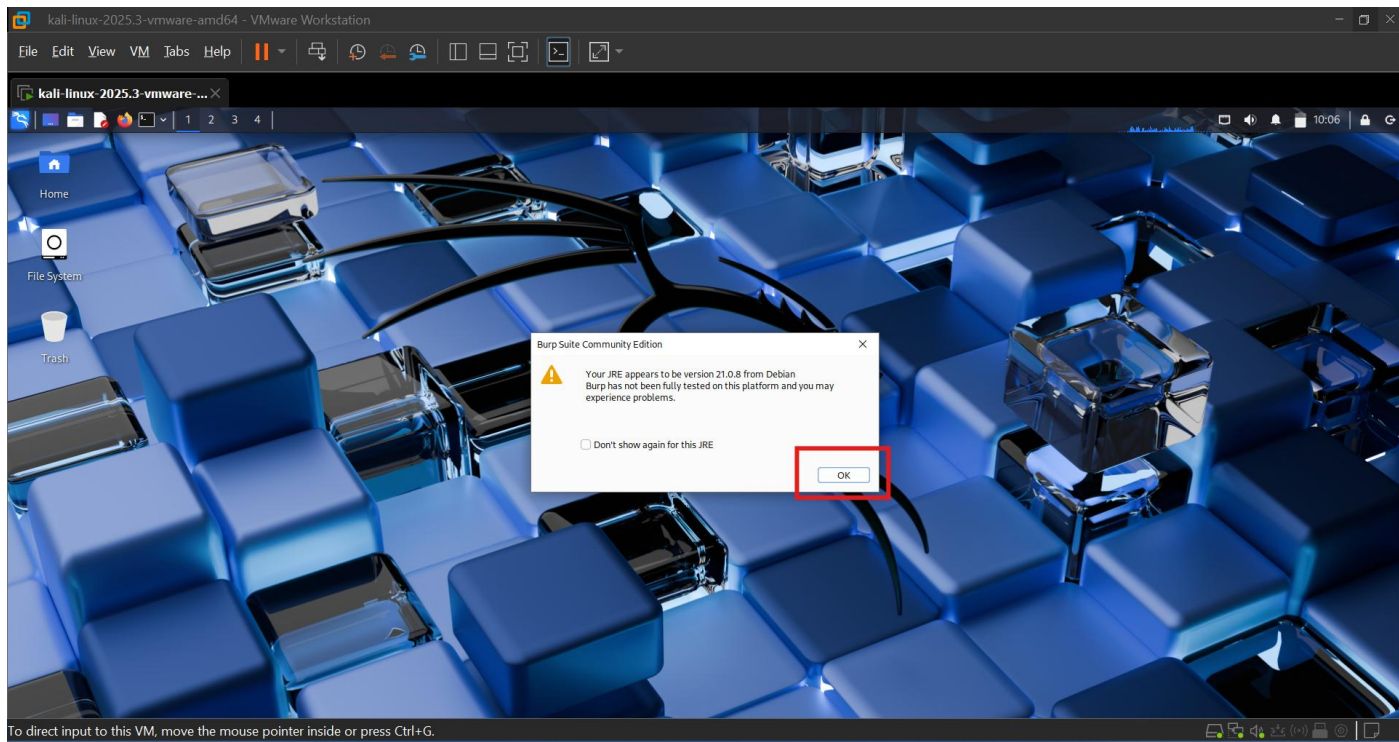




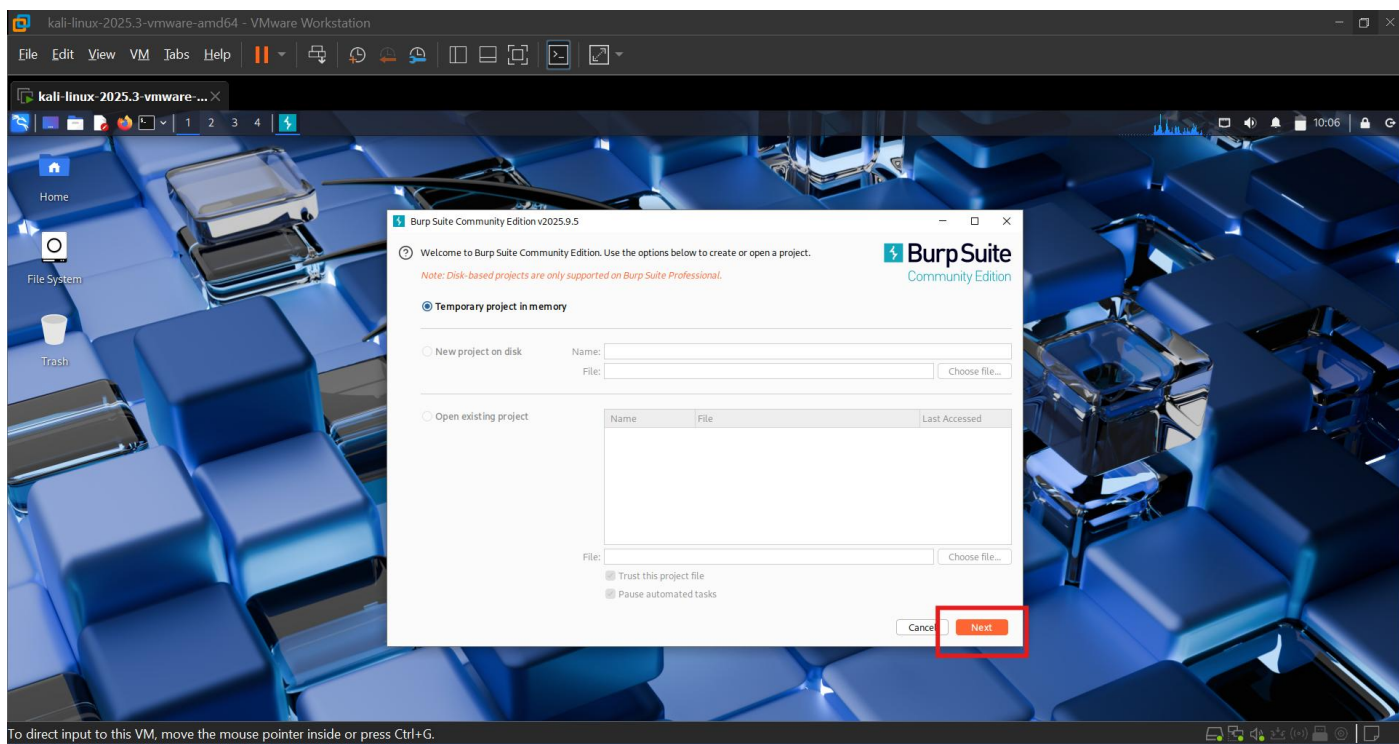
1.3 Opening burp suite application in kali machine by searching it from the list of application



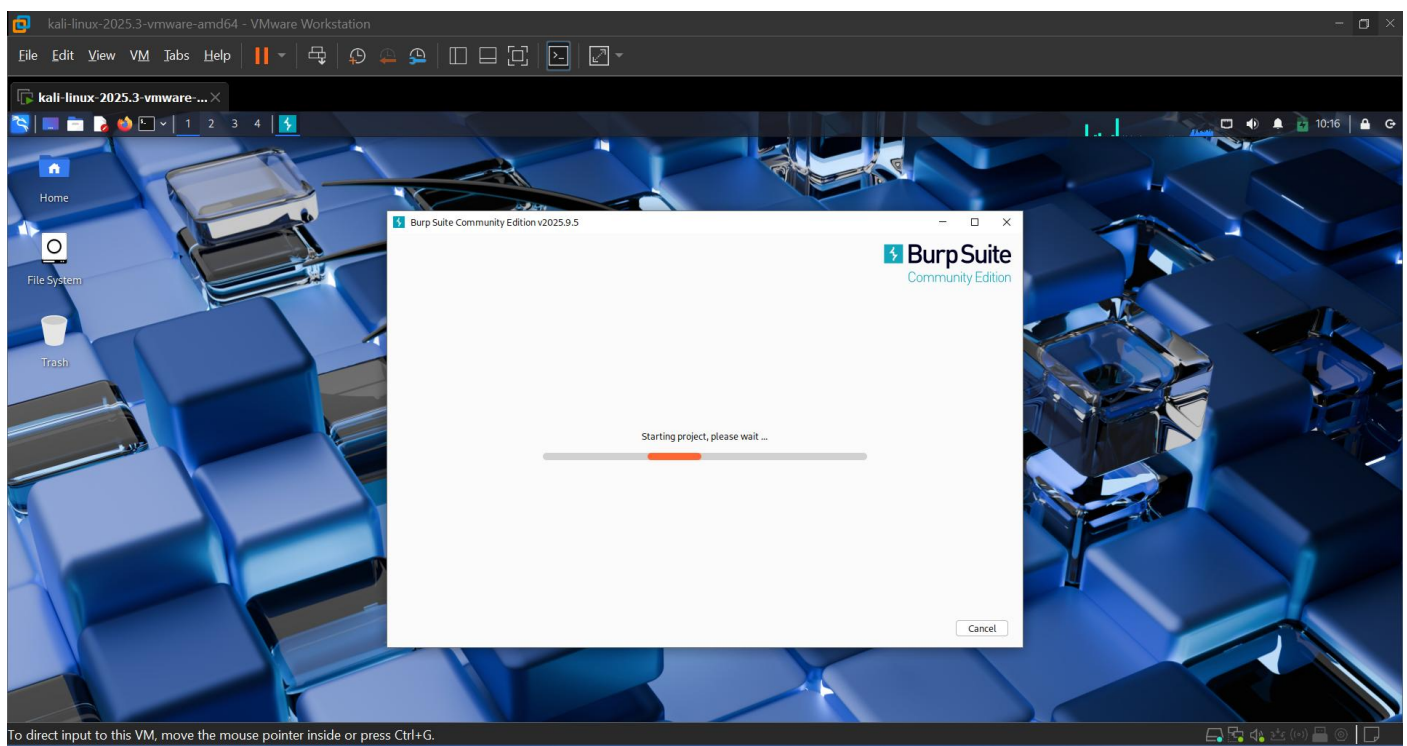
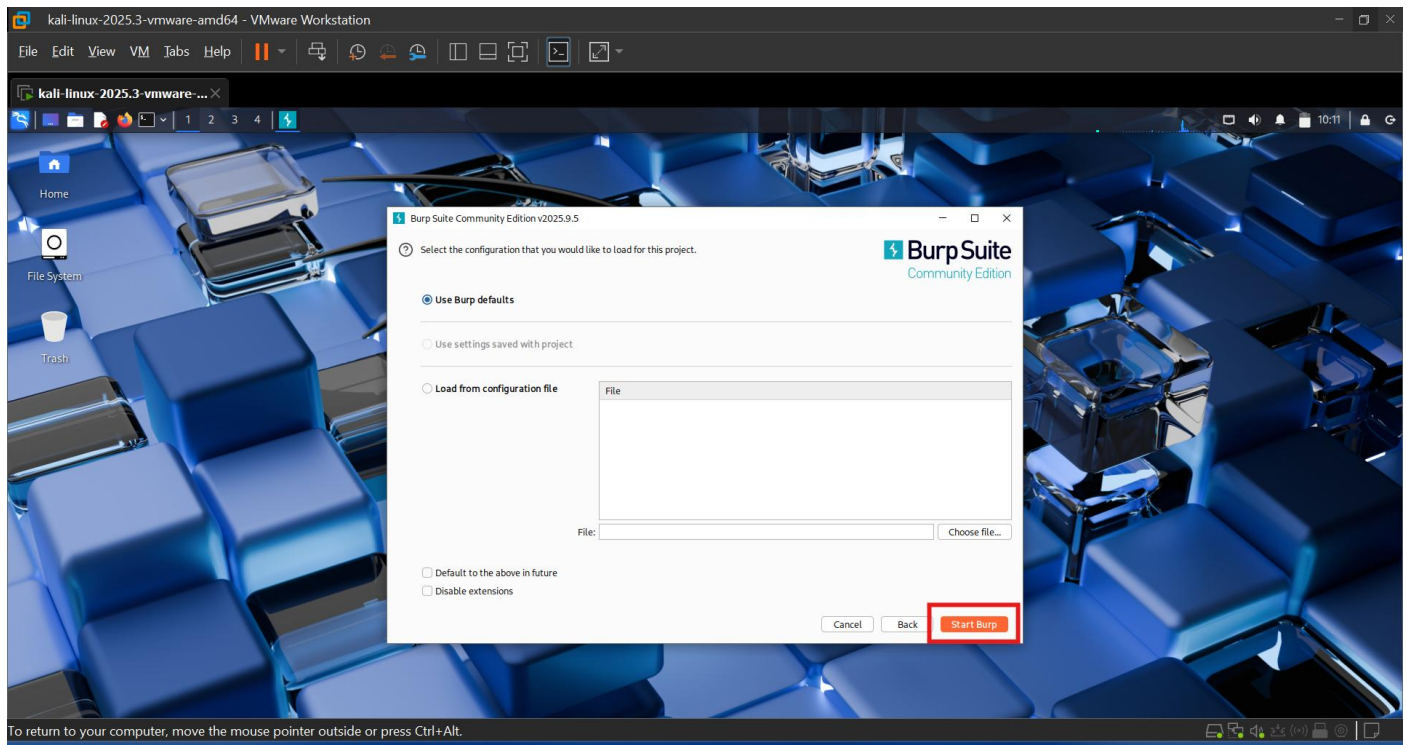
1.4 Clicked on ok



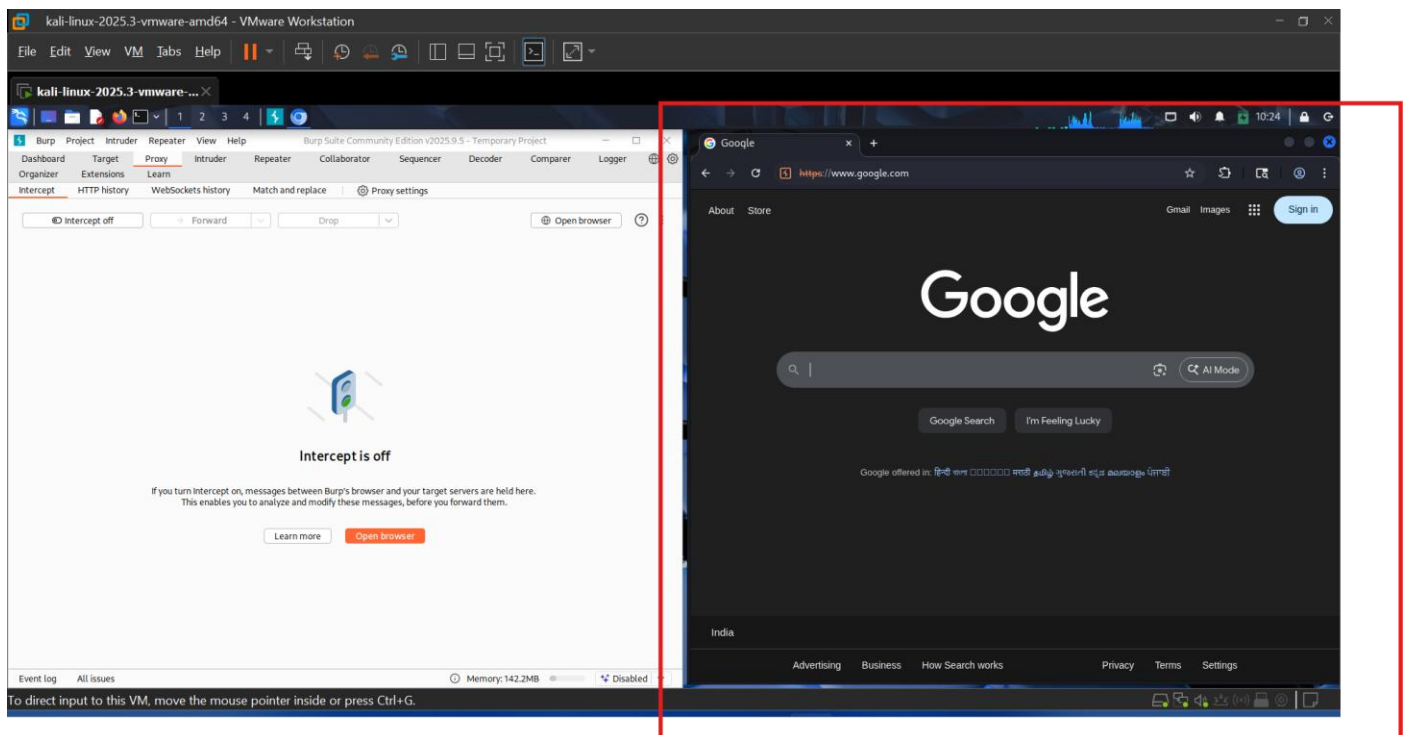
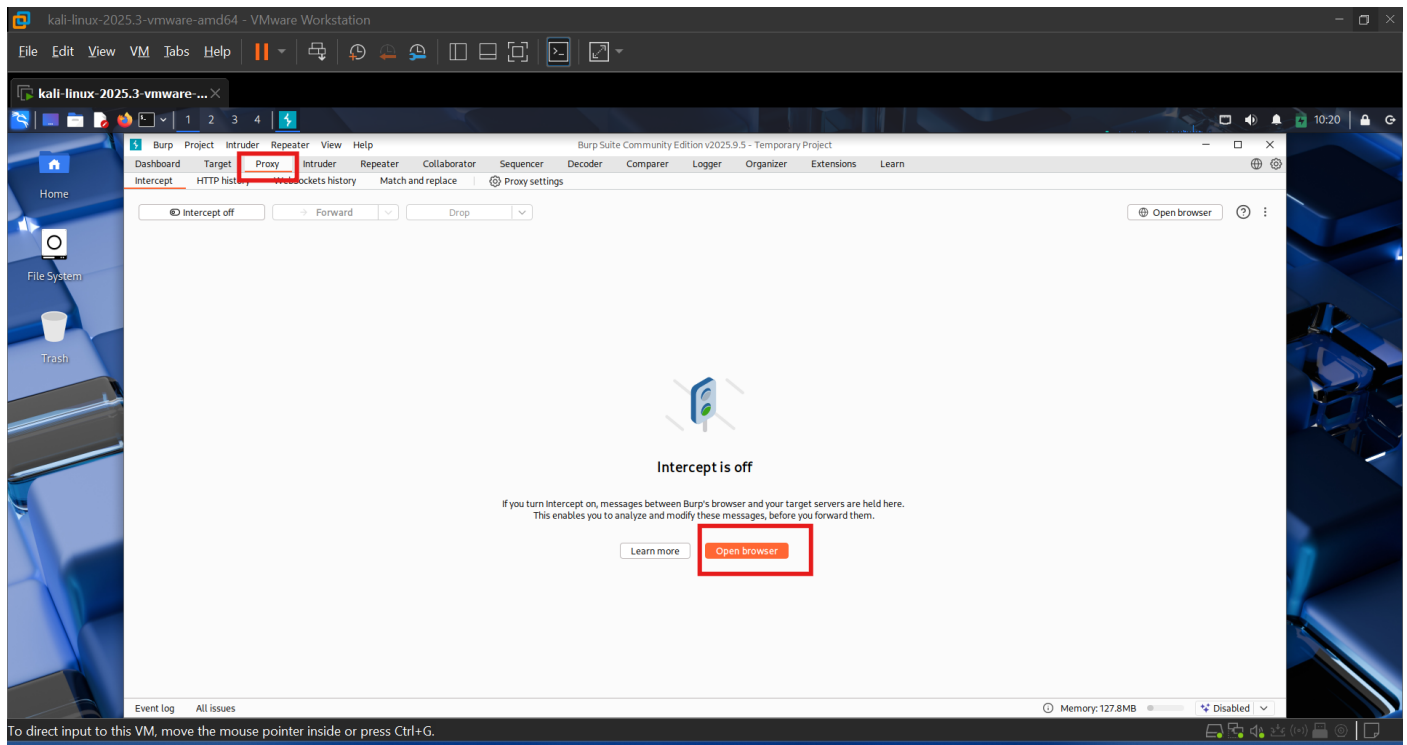
1.5 Clicked on Next



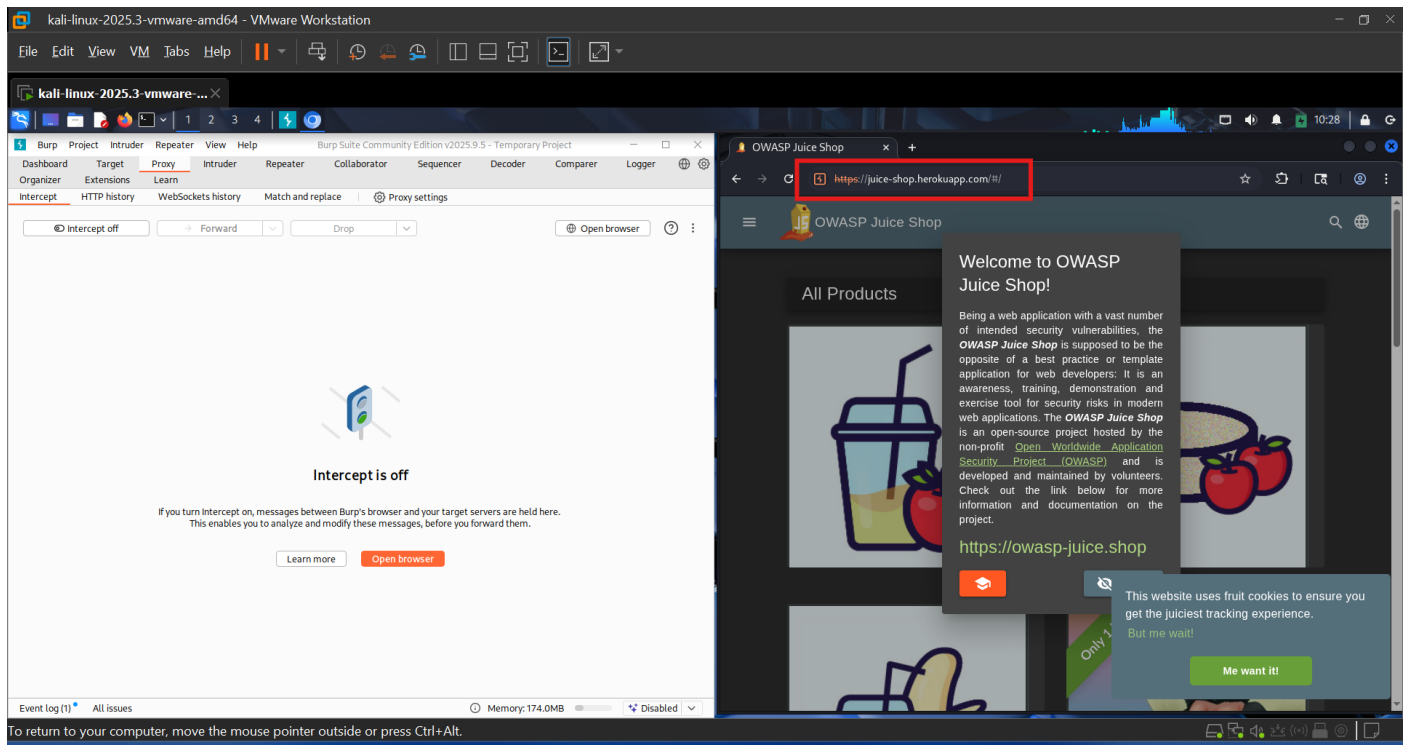
1.6 Clicked on **Start burp**



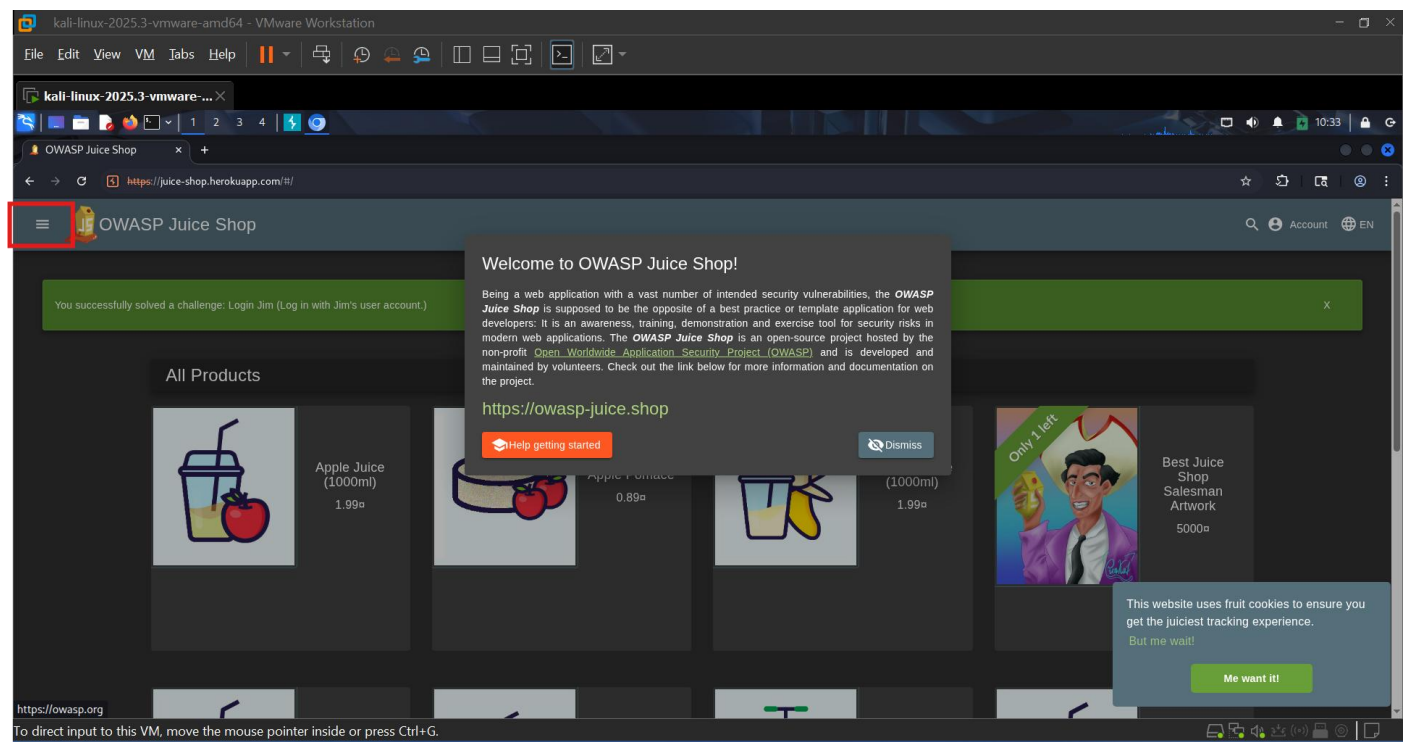
1.7 Now clicked on proxy and open browser



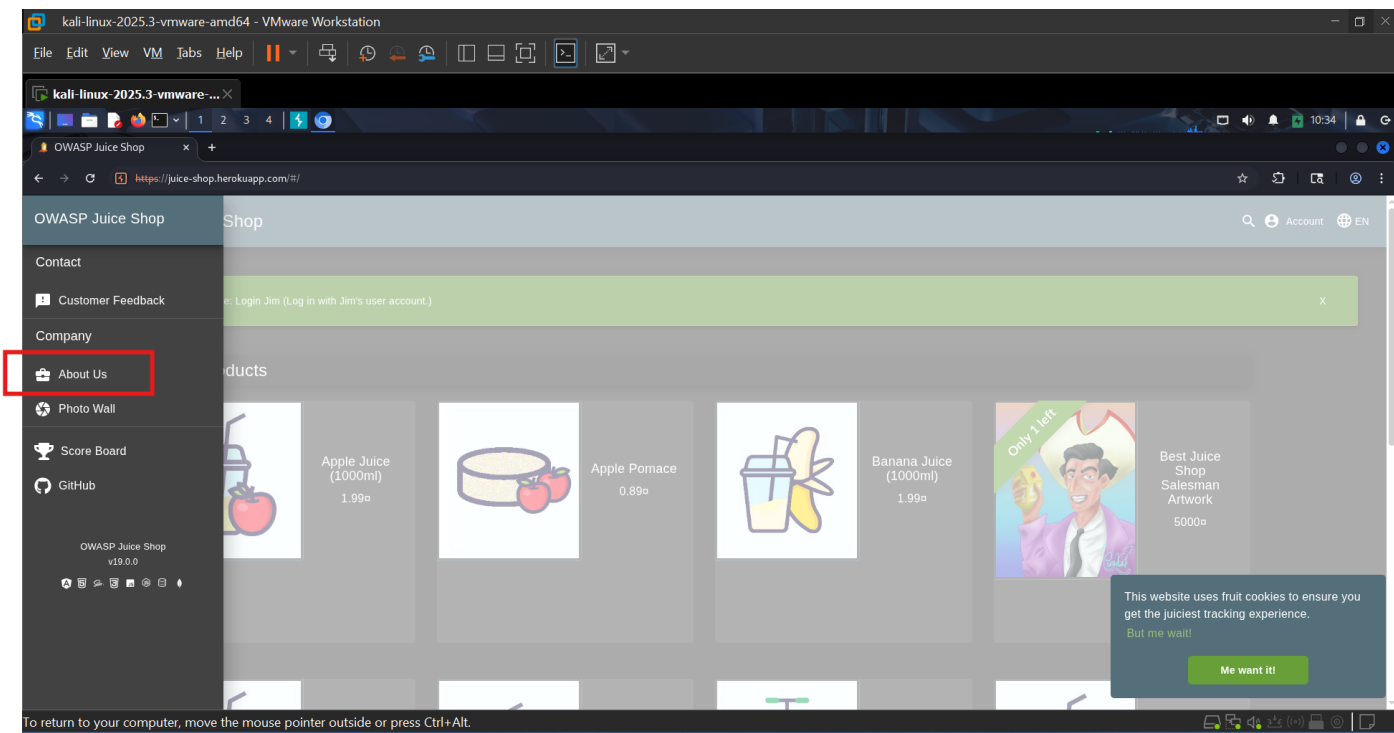
1.8 Going to this URL <https://juice-shop.herokuapp.com/#!/>



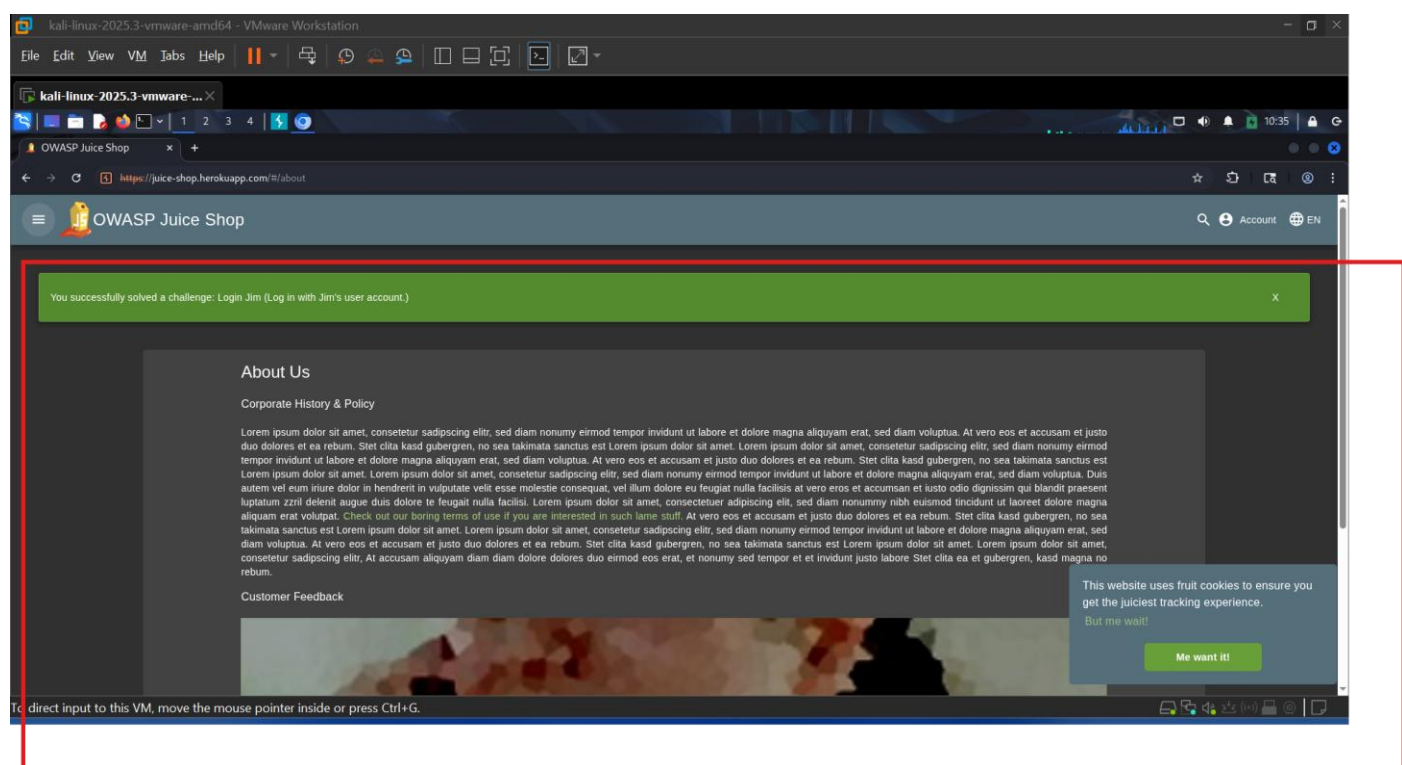
1.9 Clicked on the 3 parallel lines



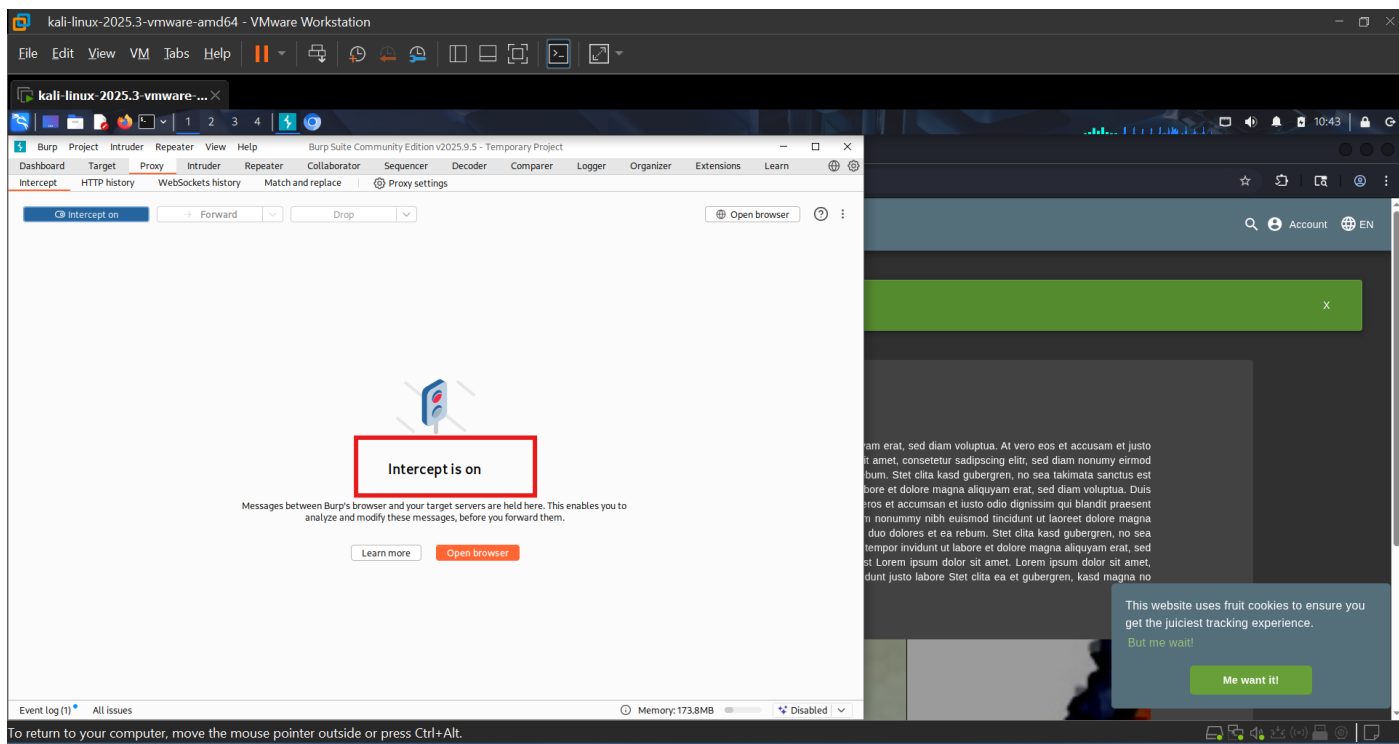
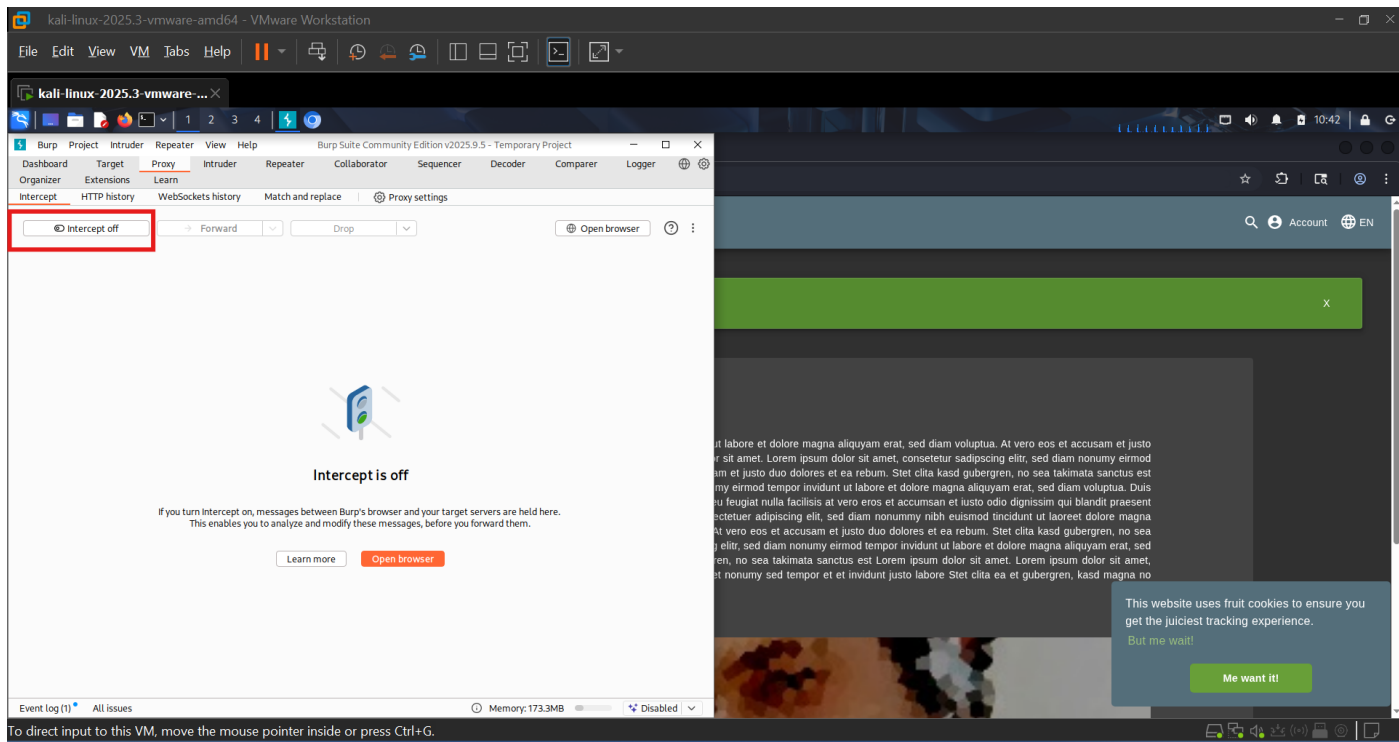
1.10 Then on About us



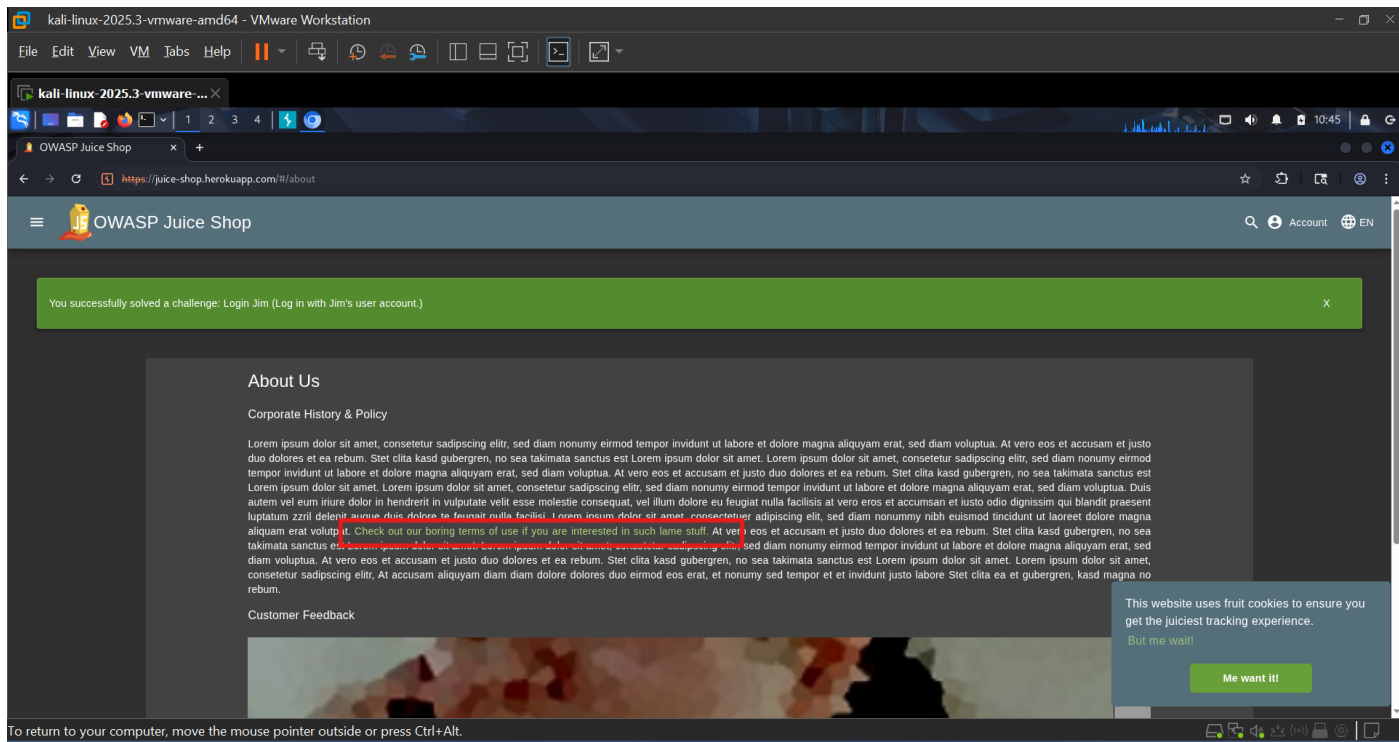
1.11 This page appeared



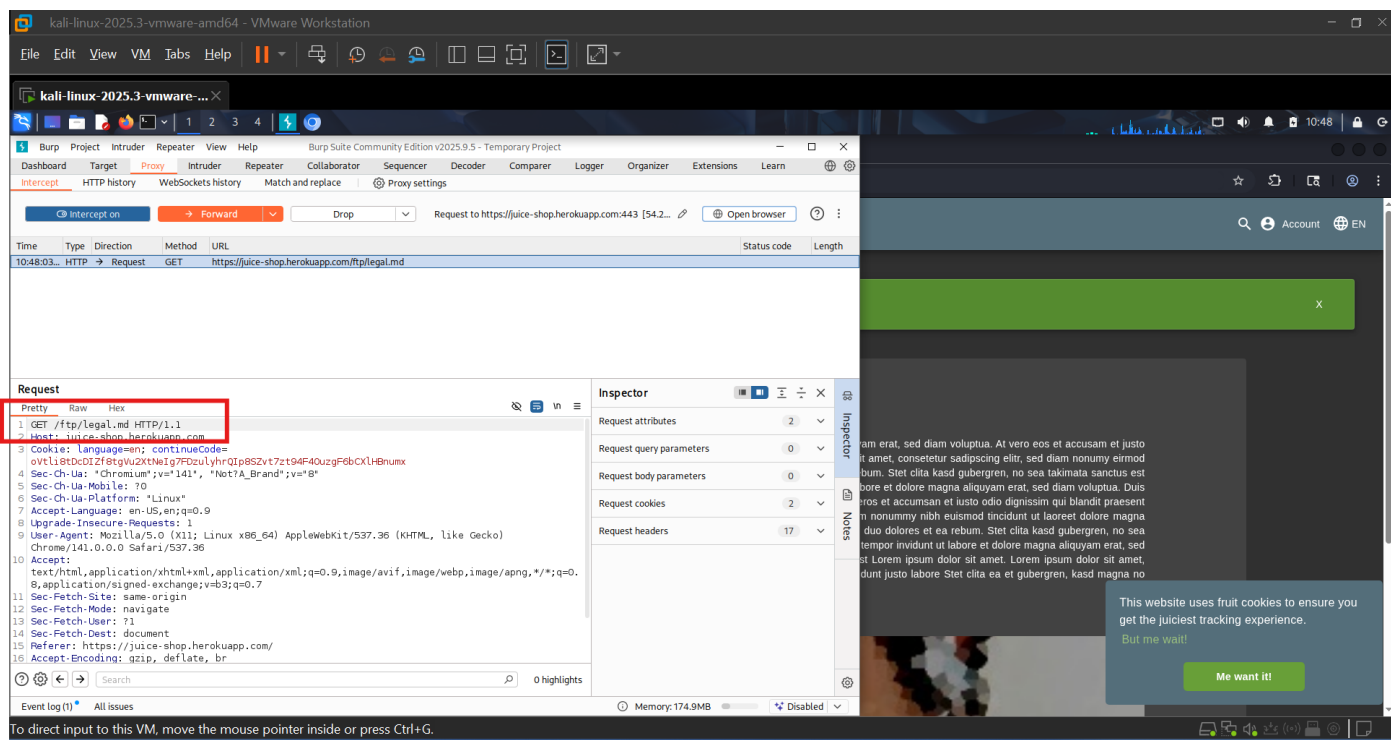
1.12 Now, turning on the intercept.



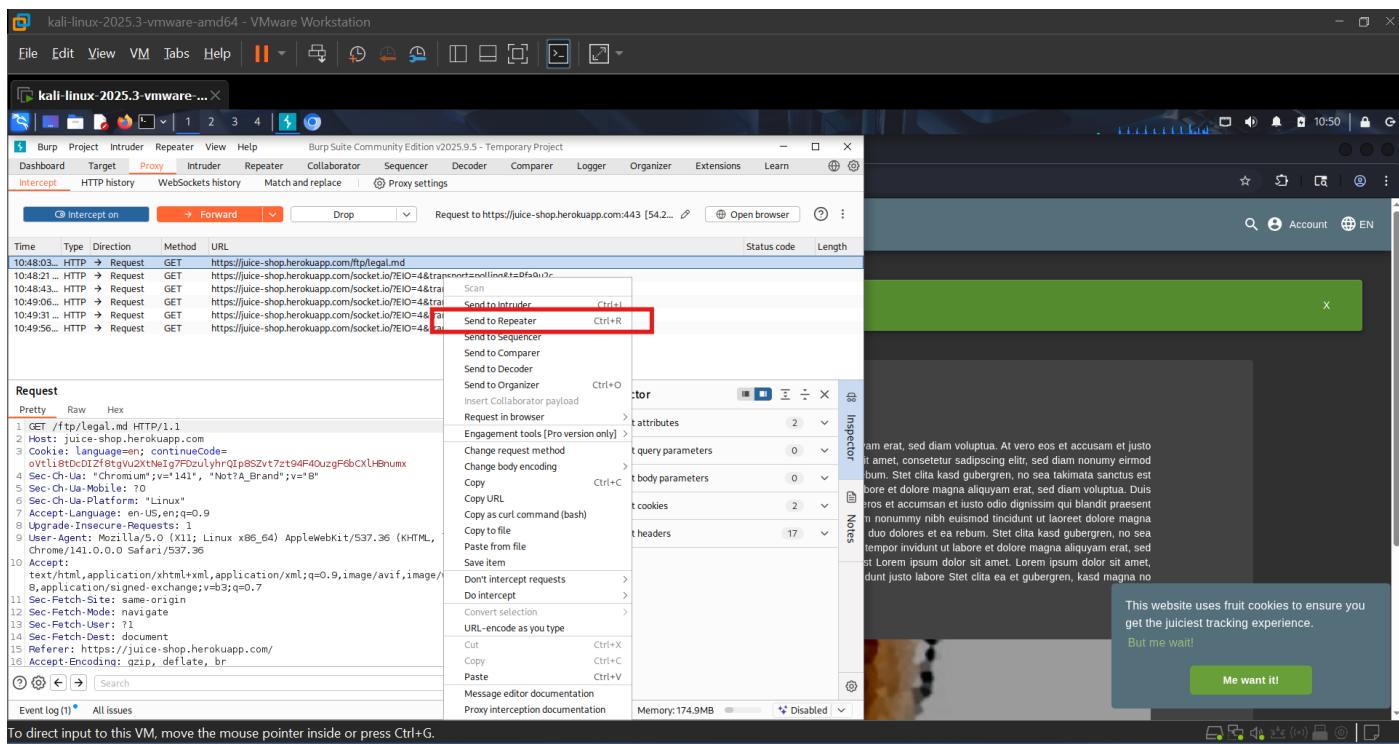
1.13 Navigating back to the about us page on the browser and clicked on “**Check out our boring terms of use if you are interested in such lame stuff.**”



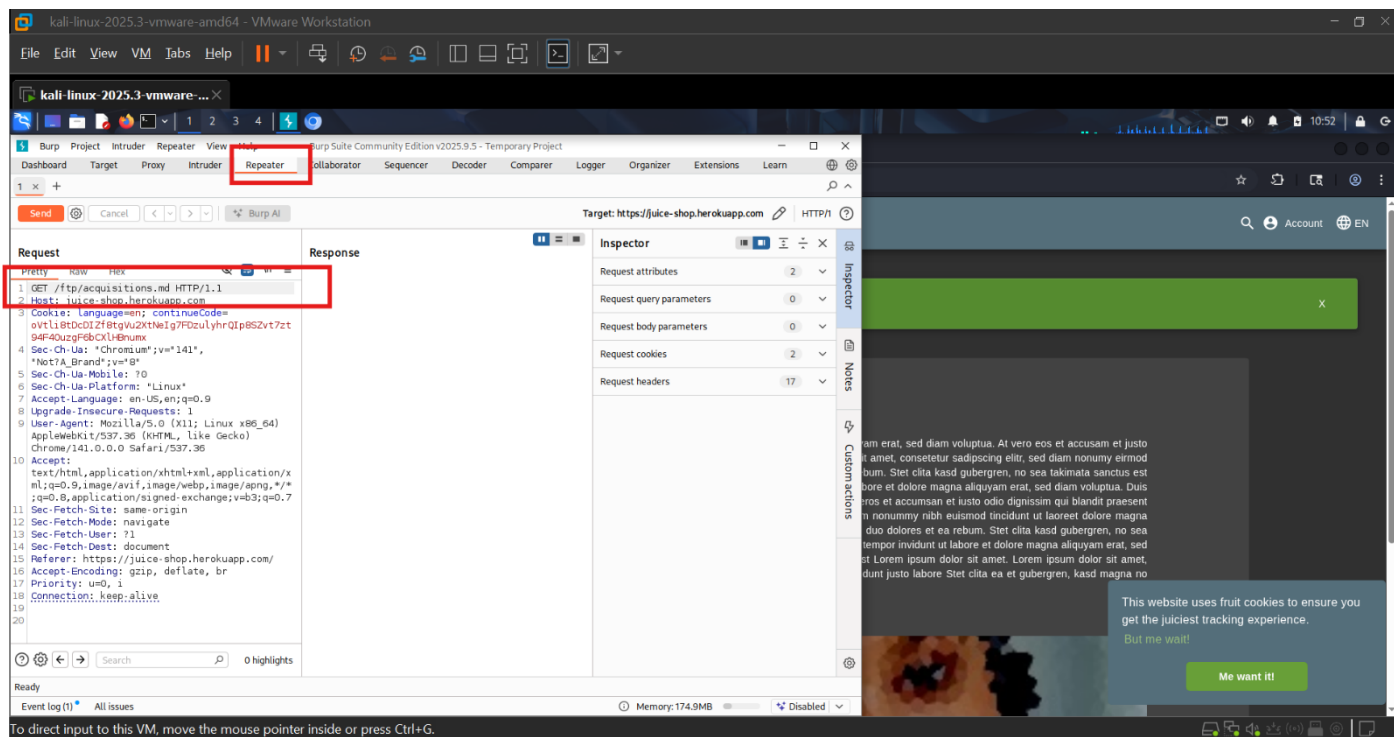
1.14 Now clicked on **Forward** until I received the heading line as “Get /ftp/legal.md HTTP/1.1”



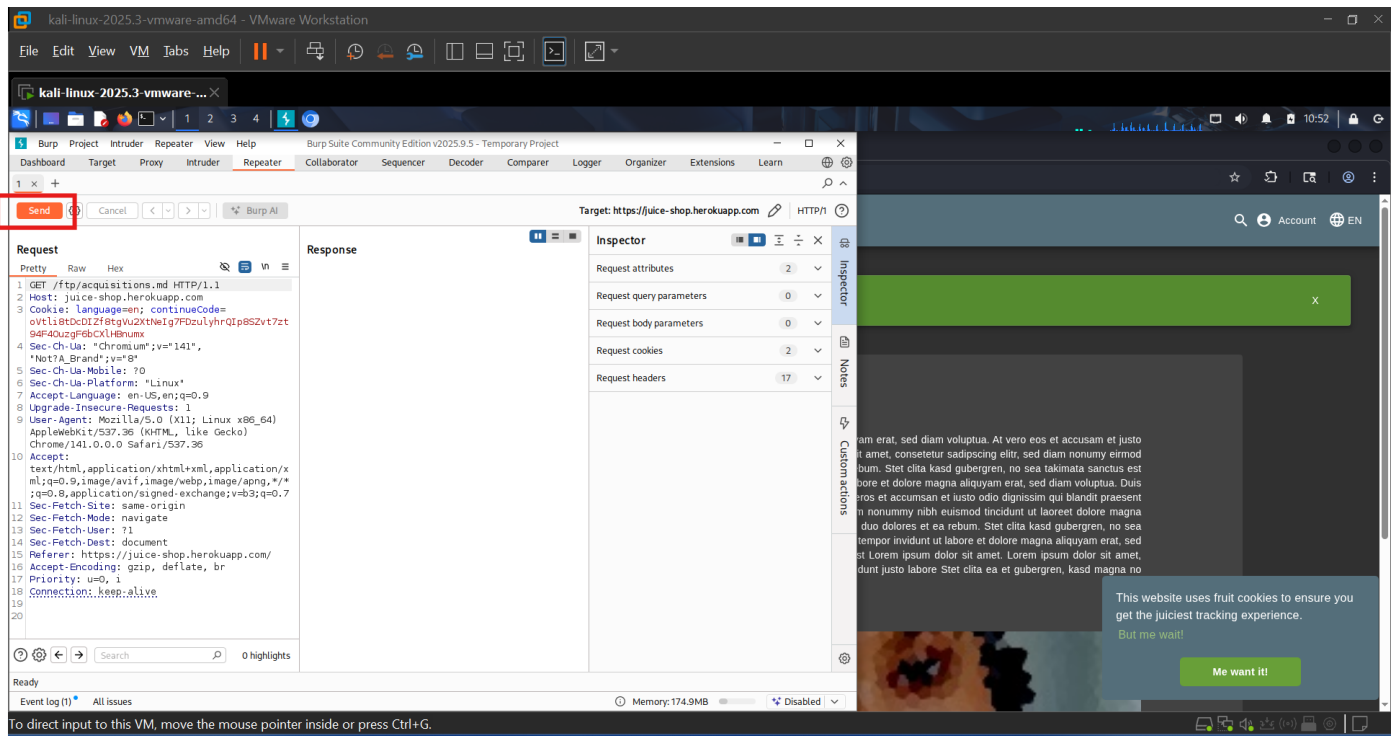
1.15 Next, right-clicked on the blank-space and clicked on “Send to Repeater”.



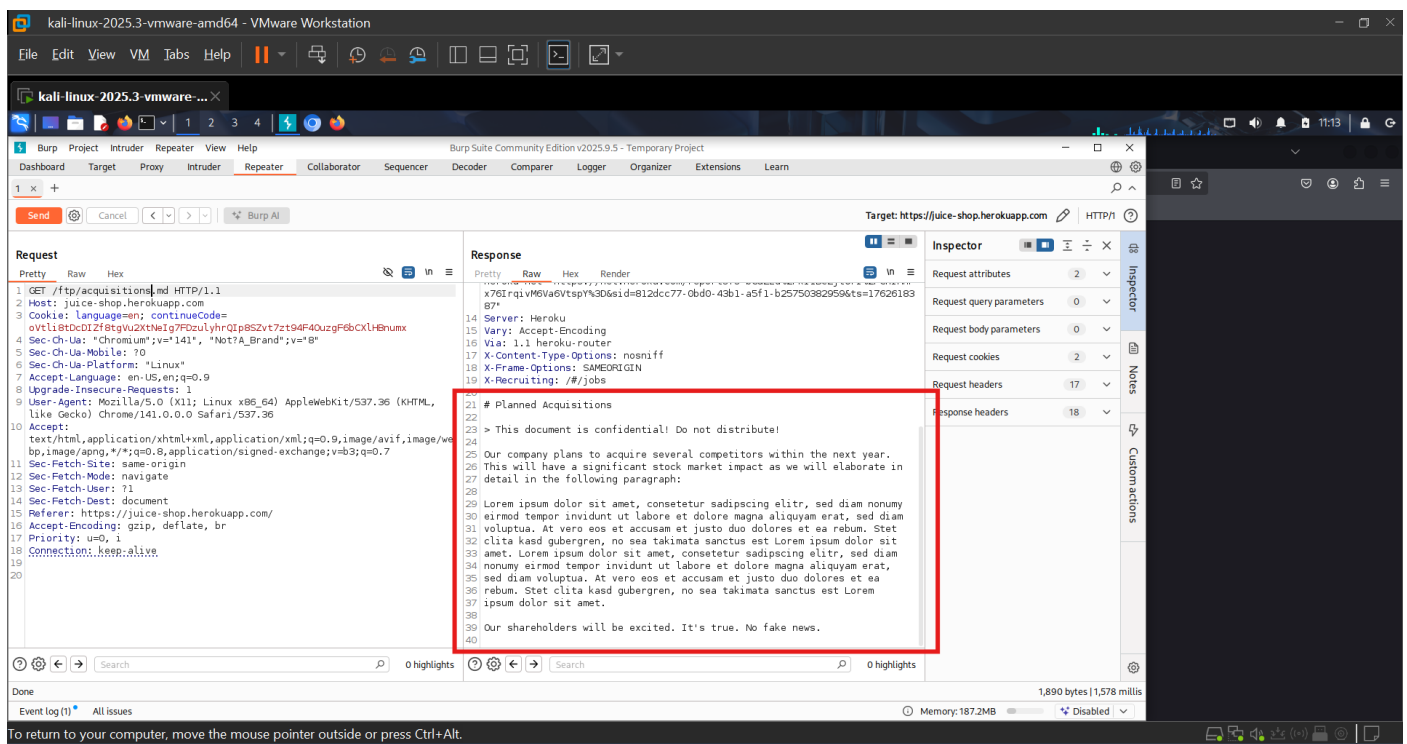
1.16 Clicked on **Repeater**, and replaced the “GET /ftp/legal.md HTTP/1.1” line with “GET /ftp/acquisitions.md HTTP/1.1”



1.17 Then clicked on **Send**.



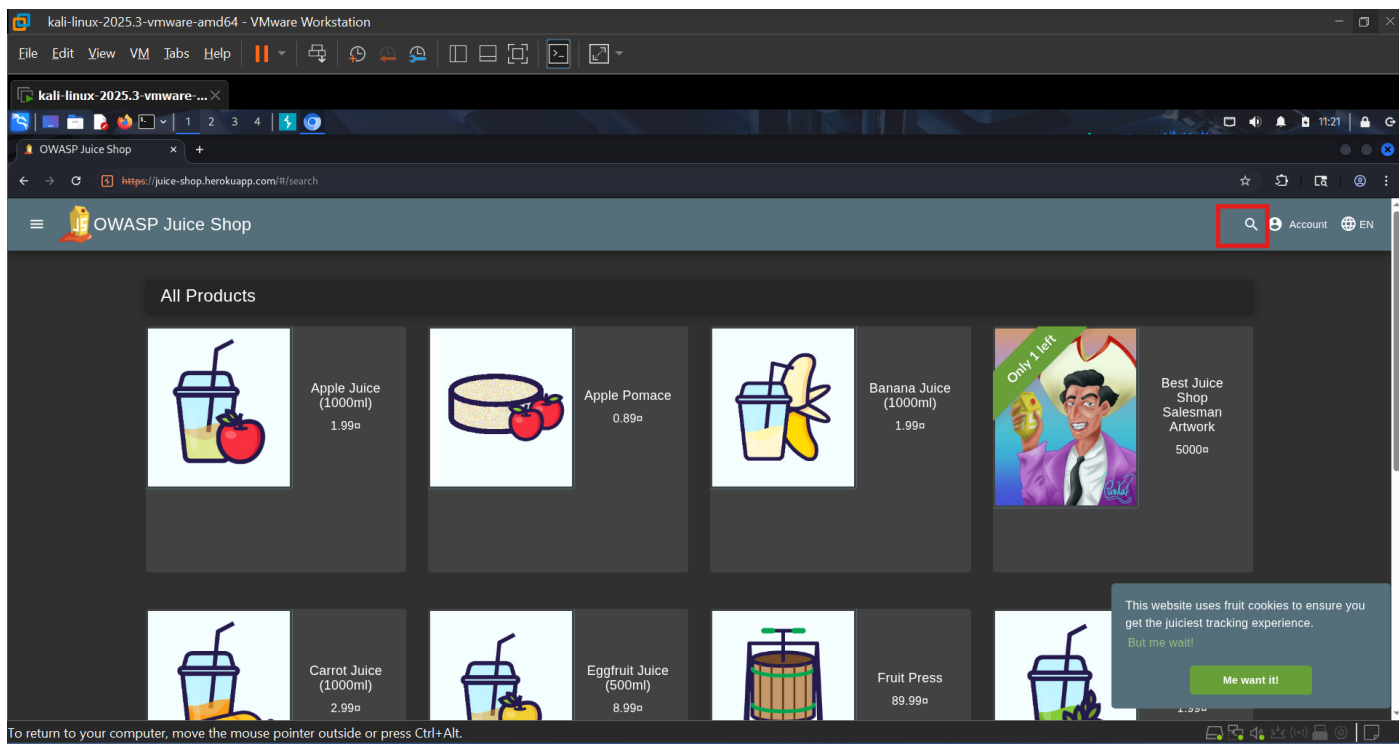
1.18 Now the sensitive file appeared after clicking on “Send” to send the tampered request.



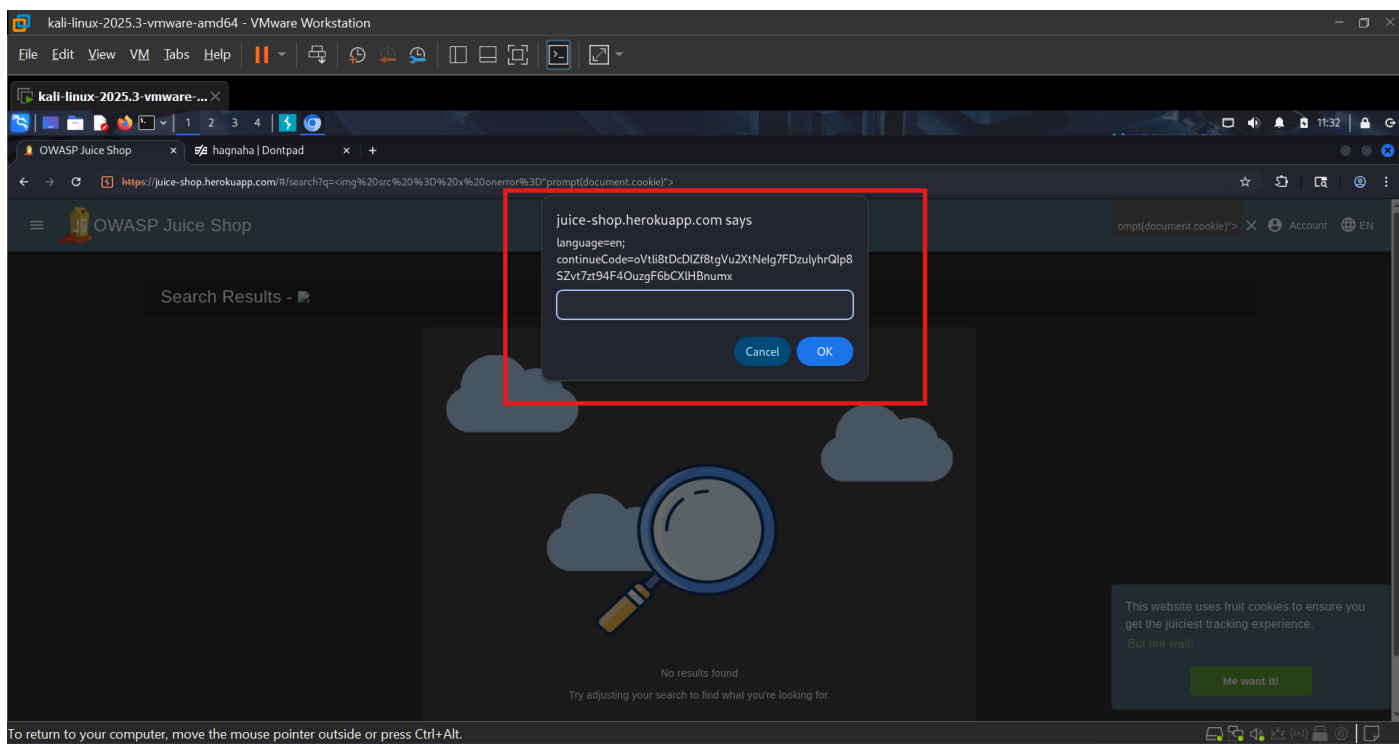
Note: The website is vulnerable to Sensitive Data Exposure.

Task 2: Exploit and test the vulnerabilities found

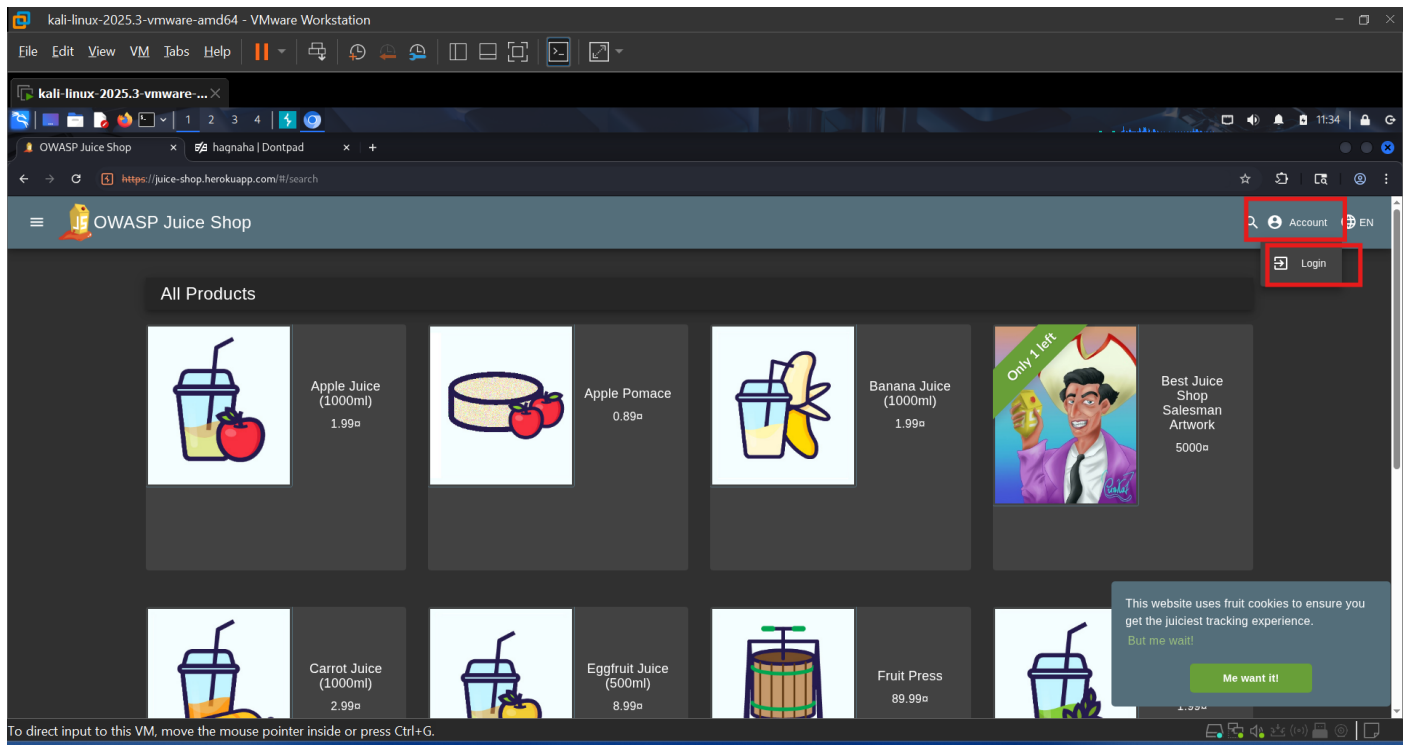
2.1 Now again shifted back to the home page of the OWASP site. Next, clicked on the “magnifying lens” icon.



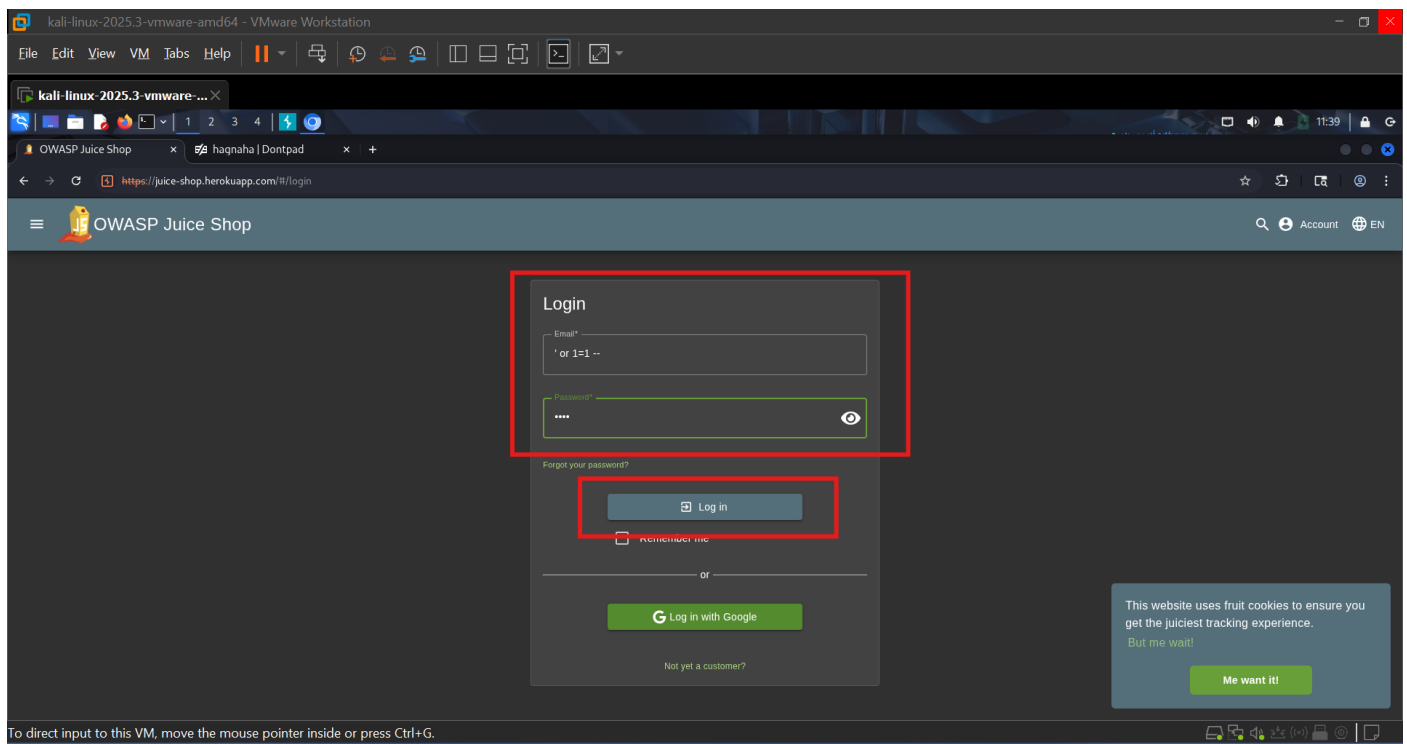
2.2 Entered the command: “” on the search bar. This command is used to check for XSS vulnerability on the website. Entering this command, the pop appears which concludes that the search field has XSS Vulnerability in it.



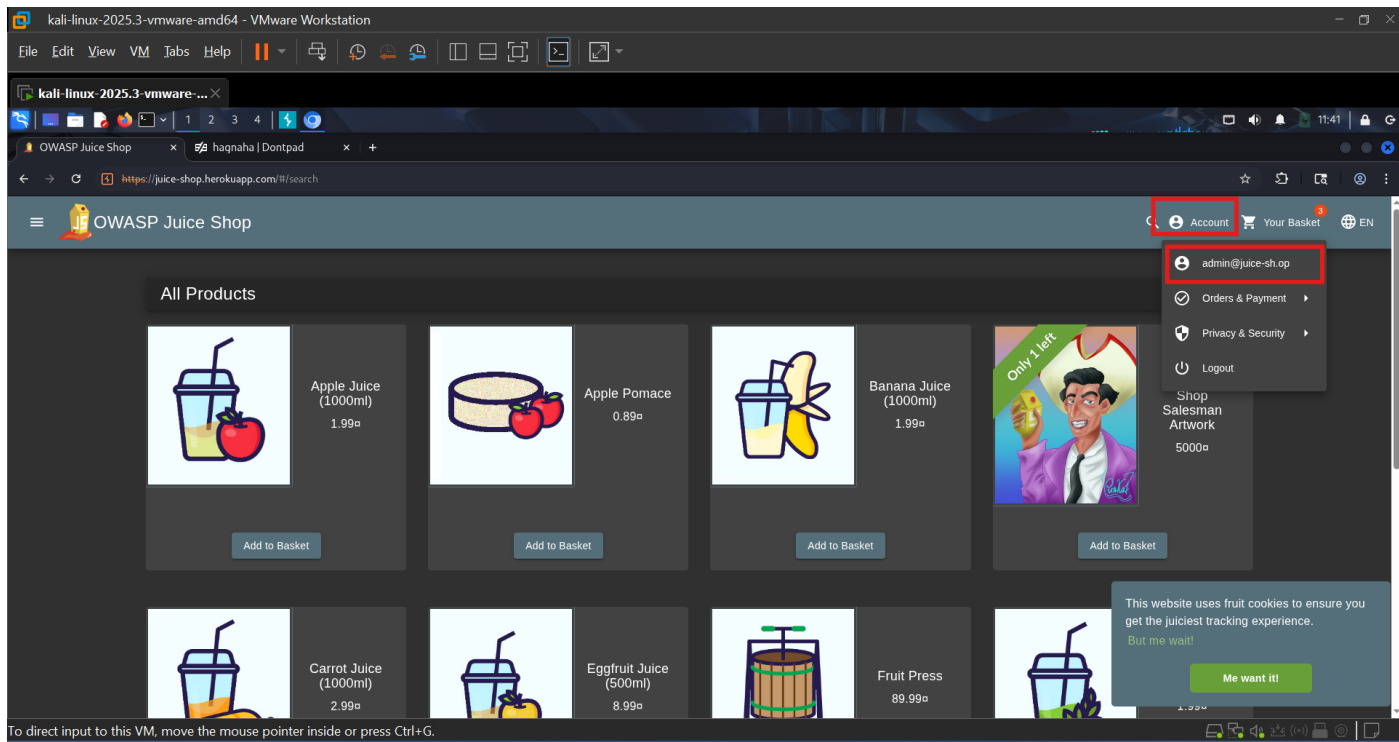
2.3 Next, going back to the home website. clicked on “Account” then clicked on “Login”.



2.4 Now trying to perform SQL injection on the login page. Under the **Email** section used the command: “**or 1=1 --**” and under the **Password** section typed “**abcd**”. Then clicked on “**Log in**”

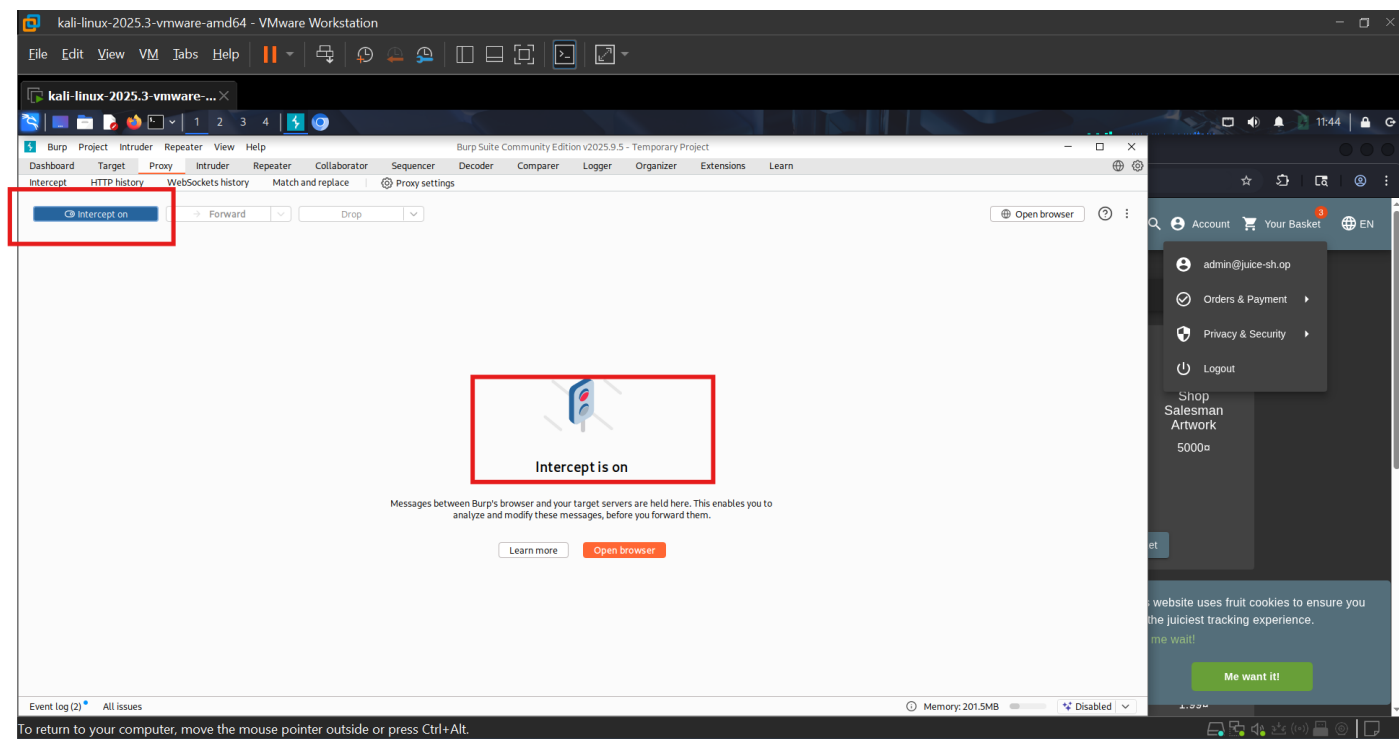


2.5 I have successfully logged in as “Admin”

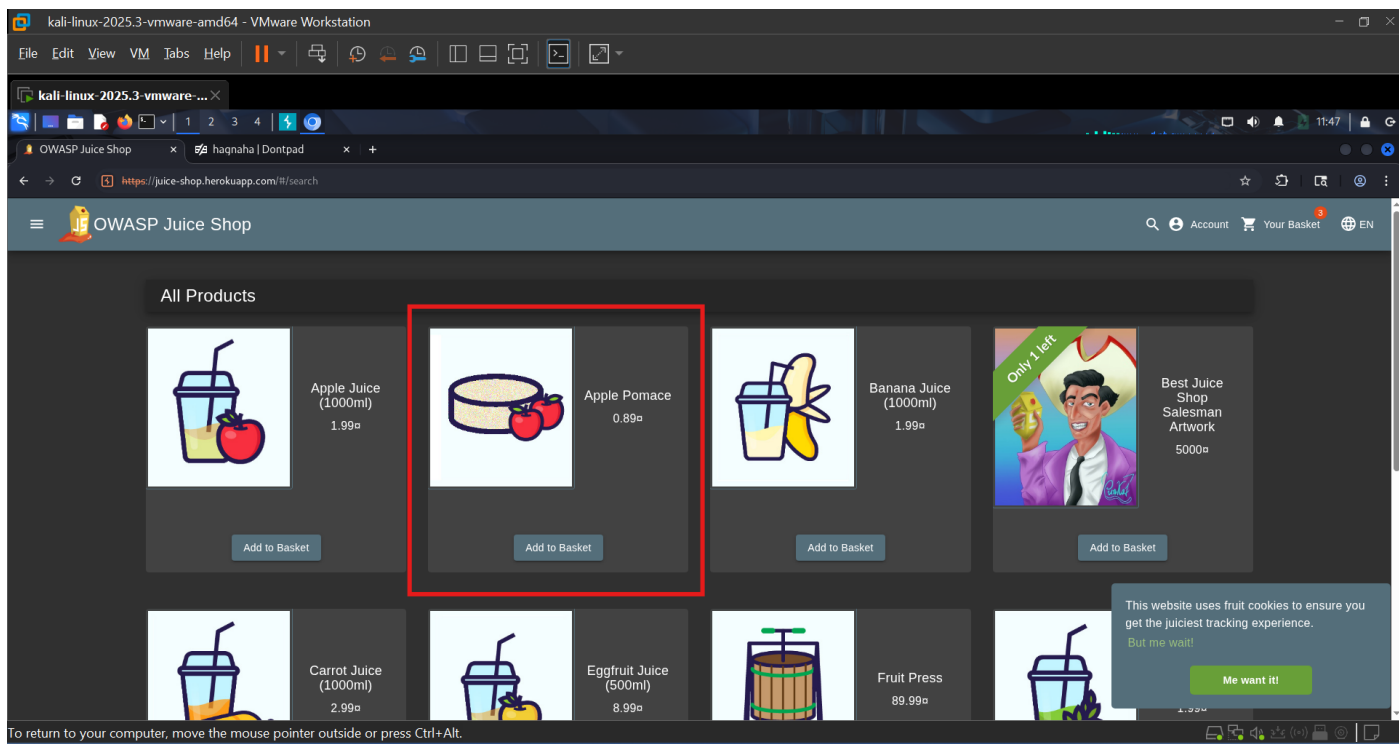


Note: This application is also vulnerable to “SQL injection”

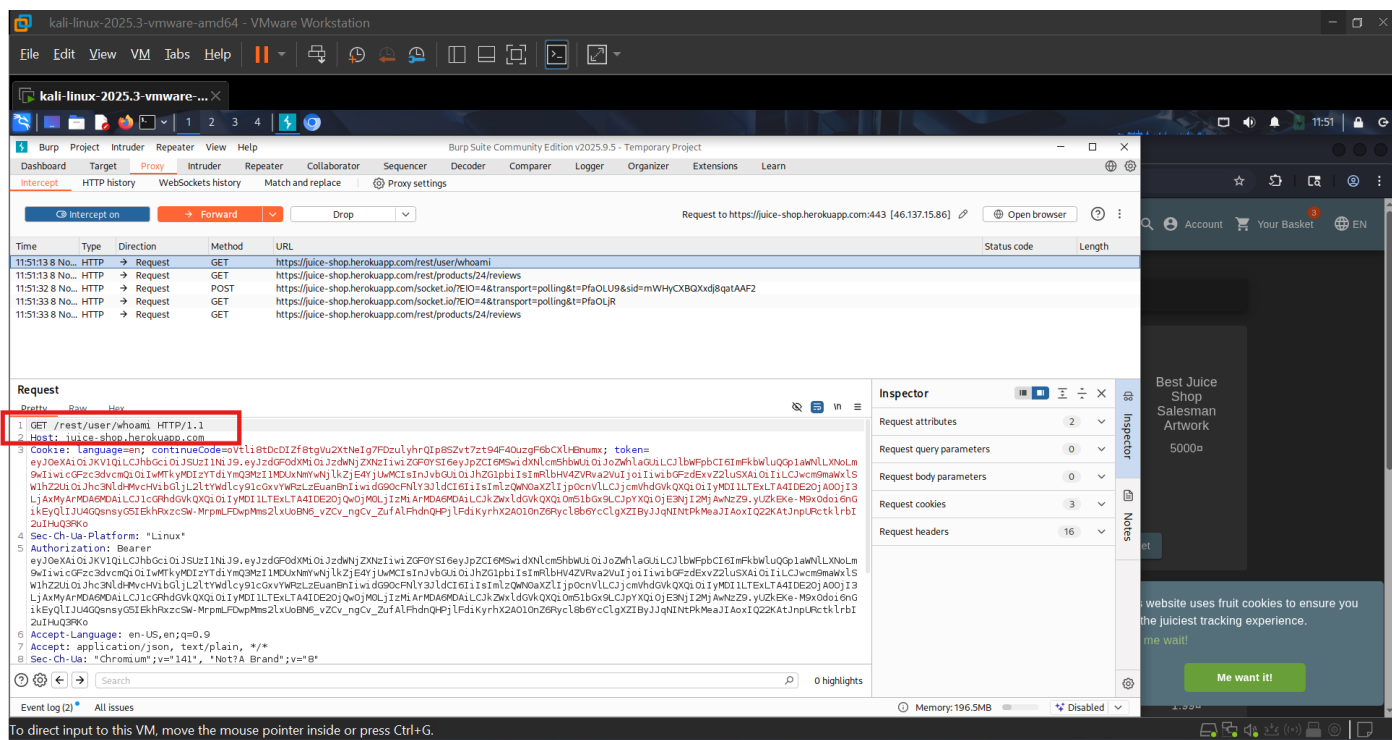
2.6 Now Turning on the intercept in the burpsuite.



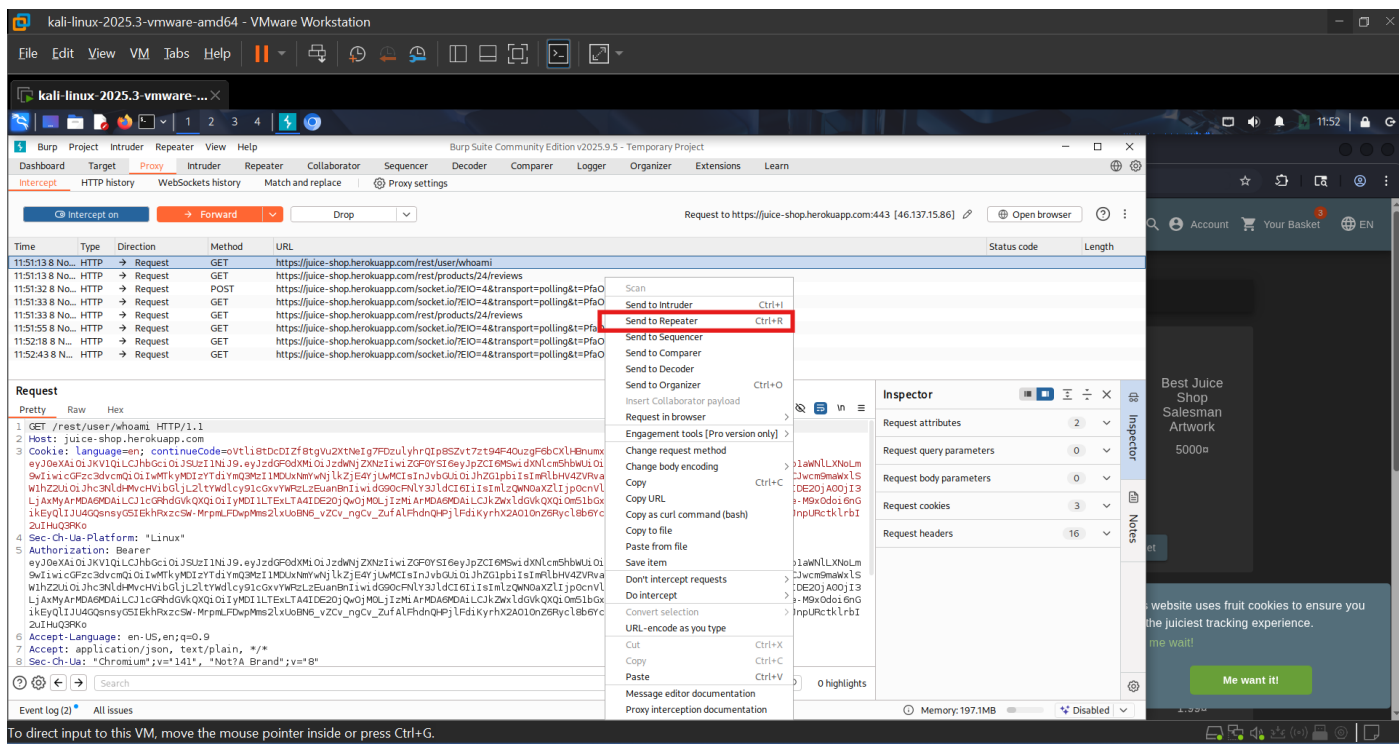
2.7 Navigating back to the browser and clicked on product.



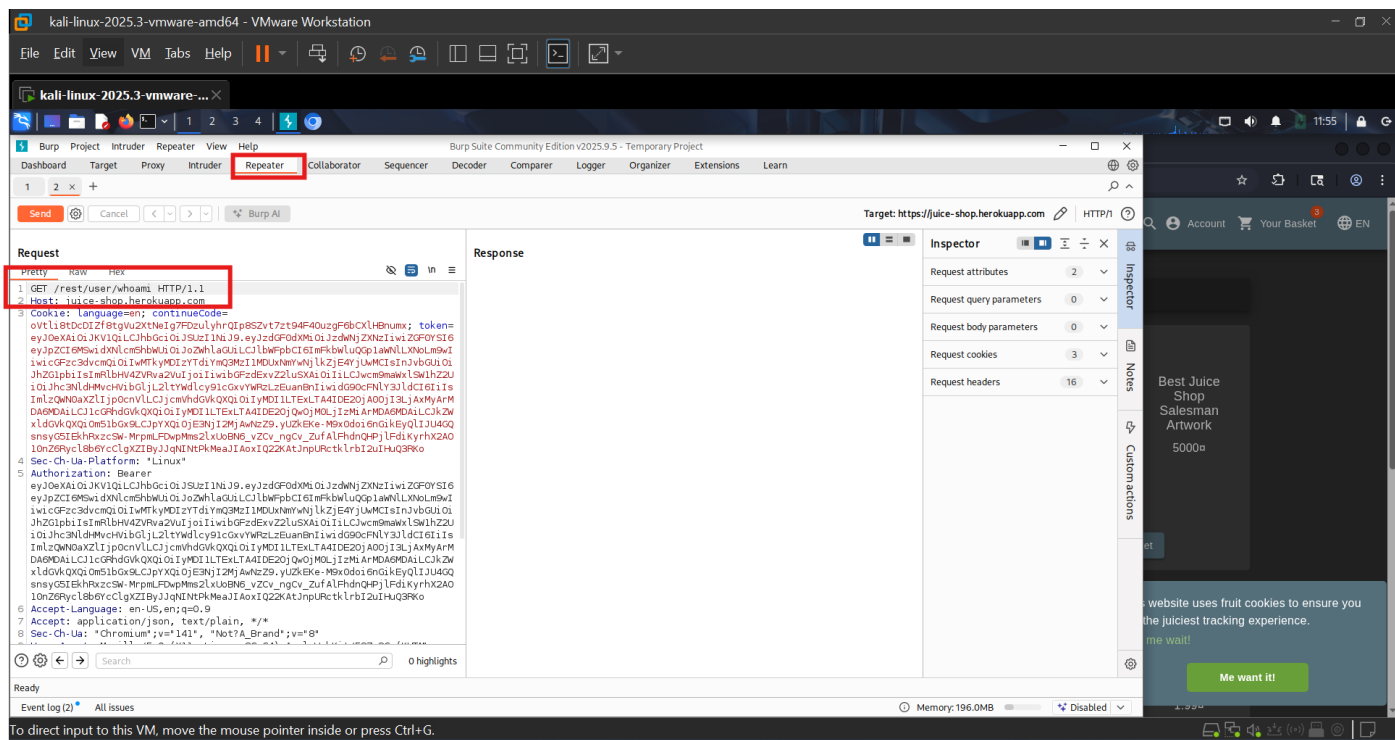
2.8 Clicked on forward until I get the heading line as “GET /rest/user/whoami HTTP/1.1”

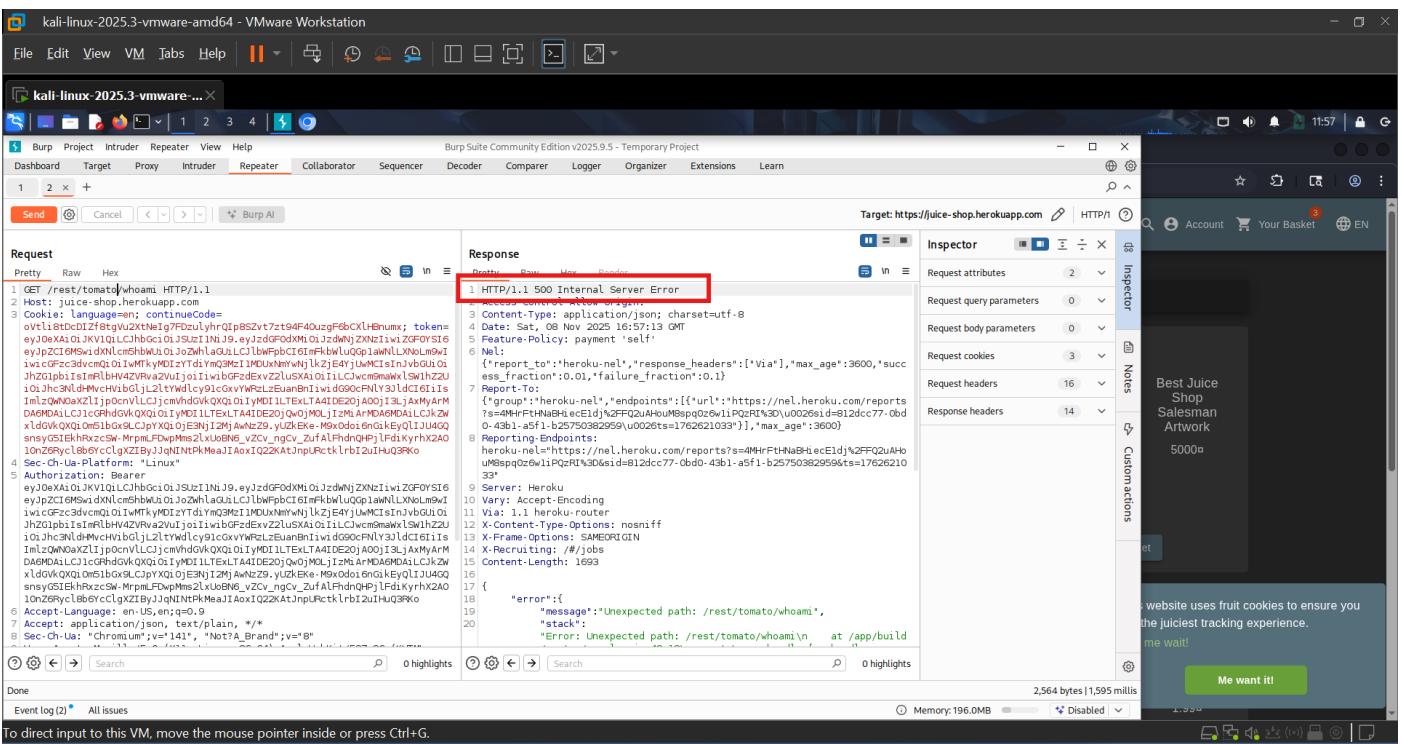
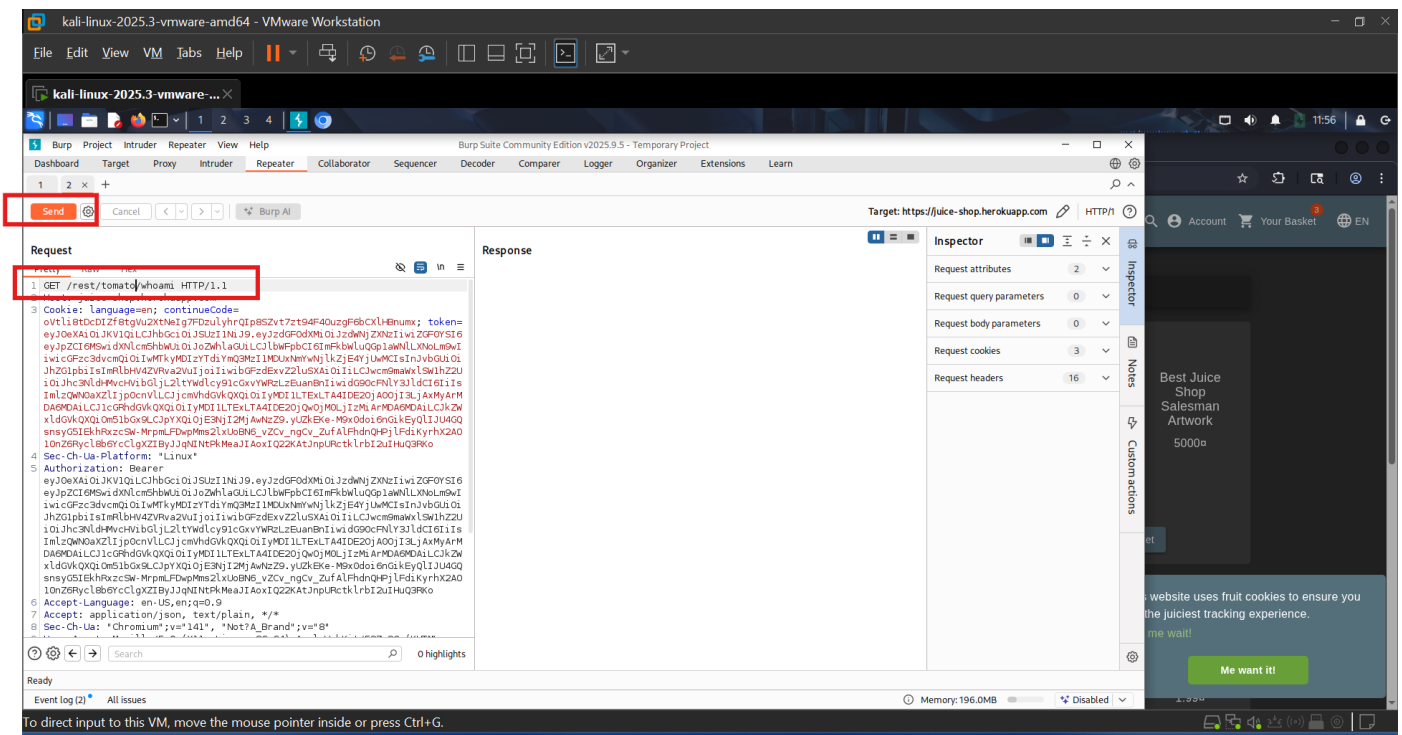


2.9 Right-clicked on the blank-space and clicked on “Send to Repeater”

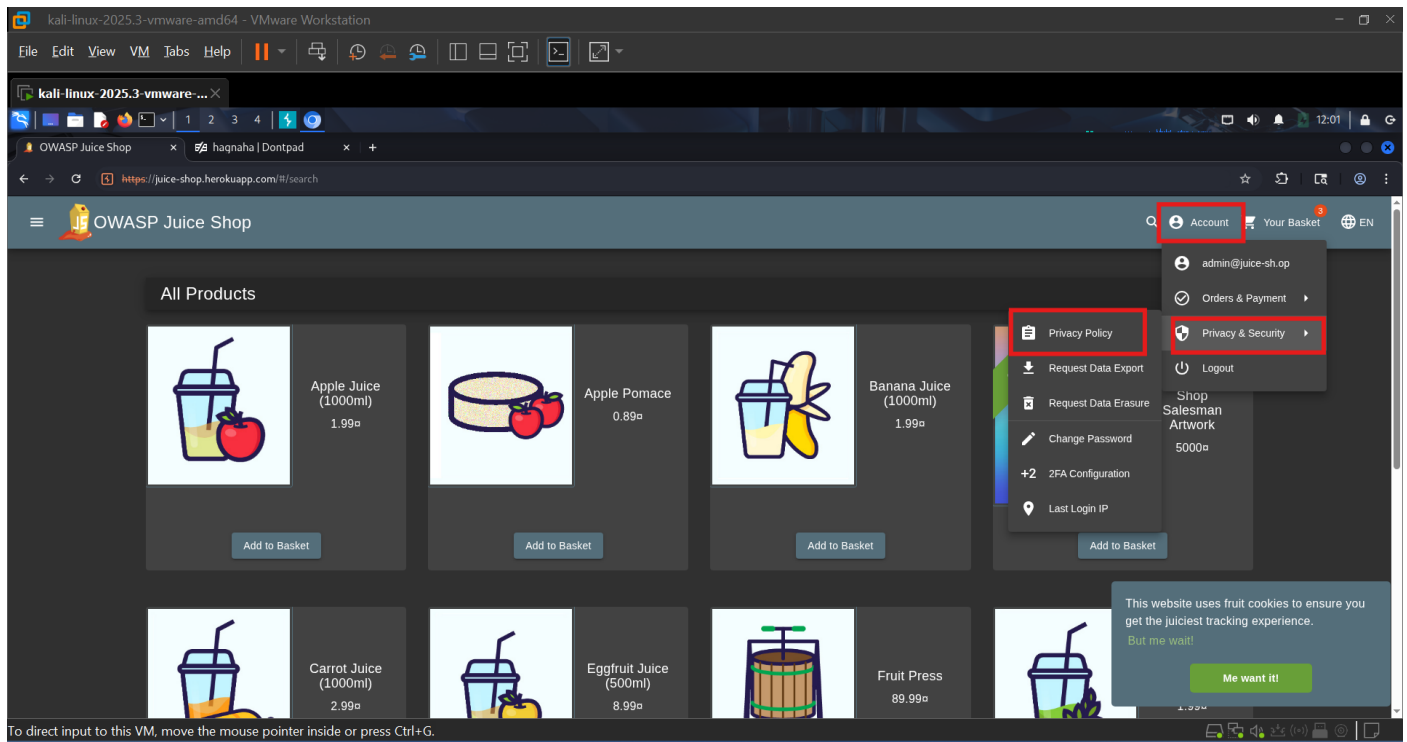


2.10 Now replaced the “/rest/user/whoami” line with “/rest/tomato” then clicked “Send”. You will see an Internal Server Error occurs which states that this website has a Poor Error Handling Vulnerability.

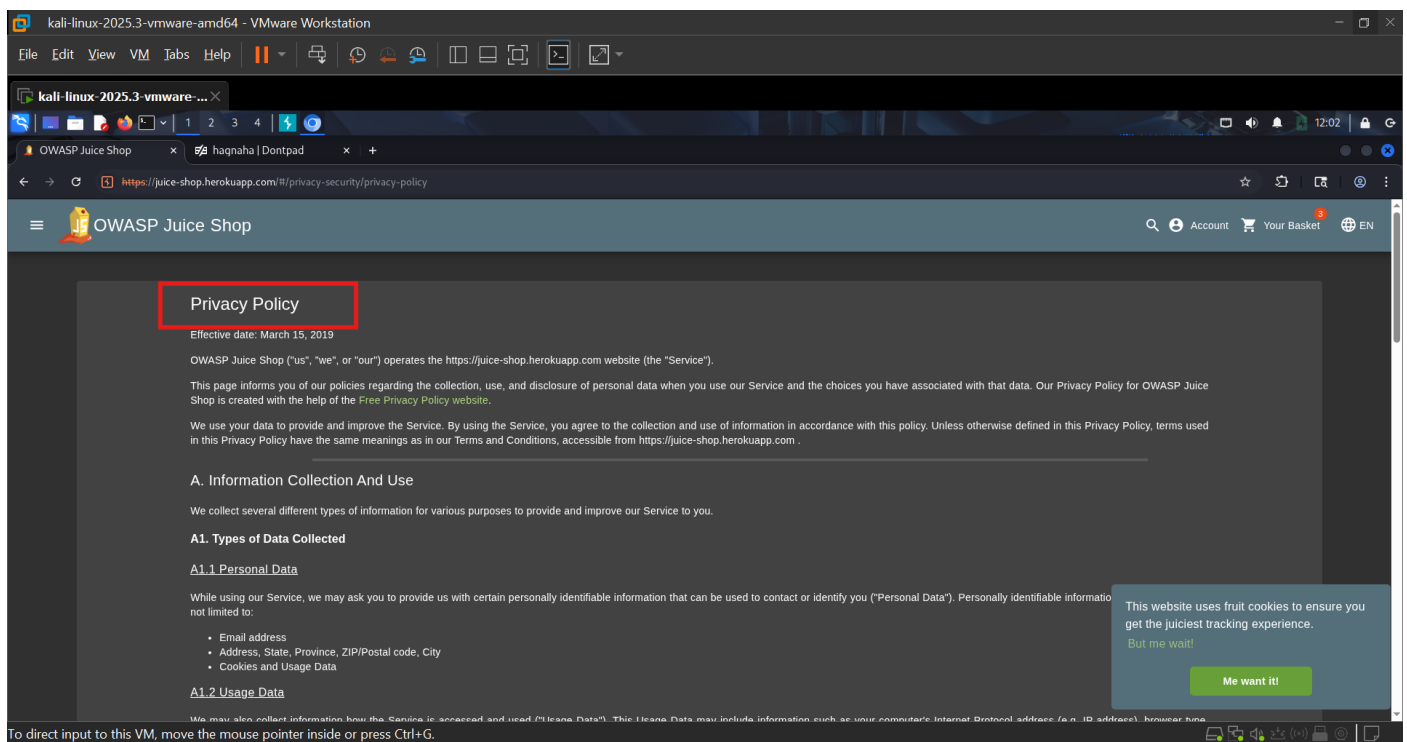




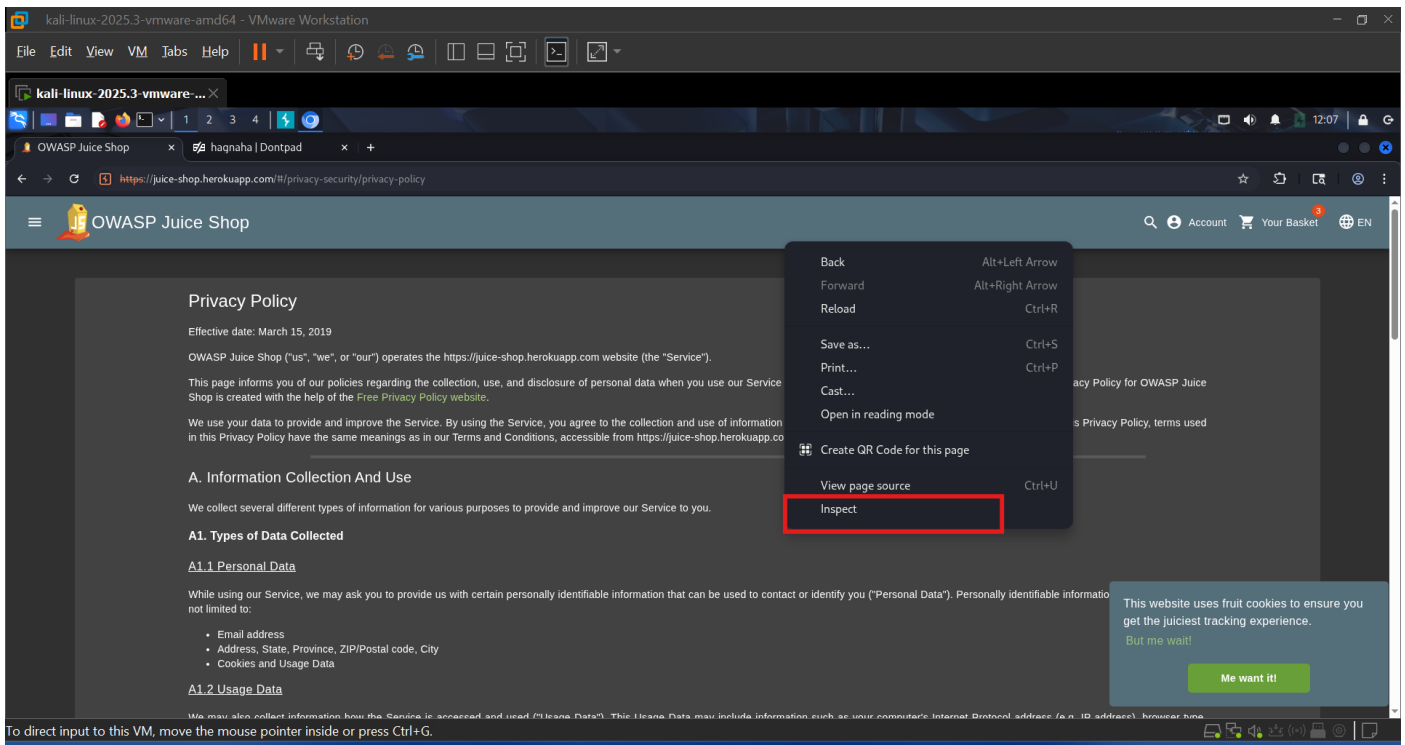
2.11 Next, returned to the website where I already logged into the “admin” account. Clicked on “Account” then “Privacy and Security” option and finally clicked on “Privacy Policy”.



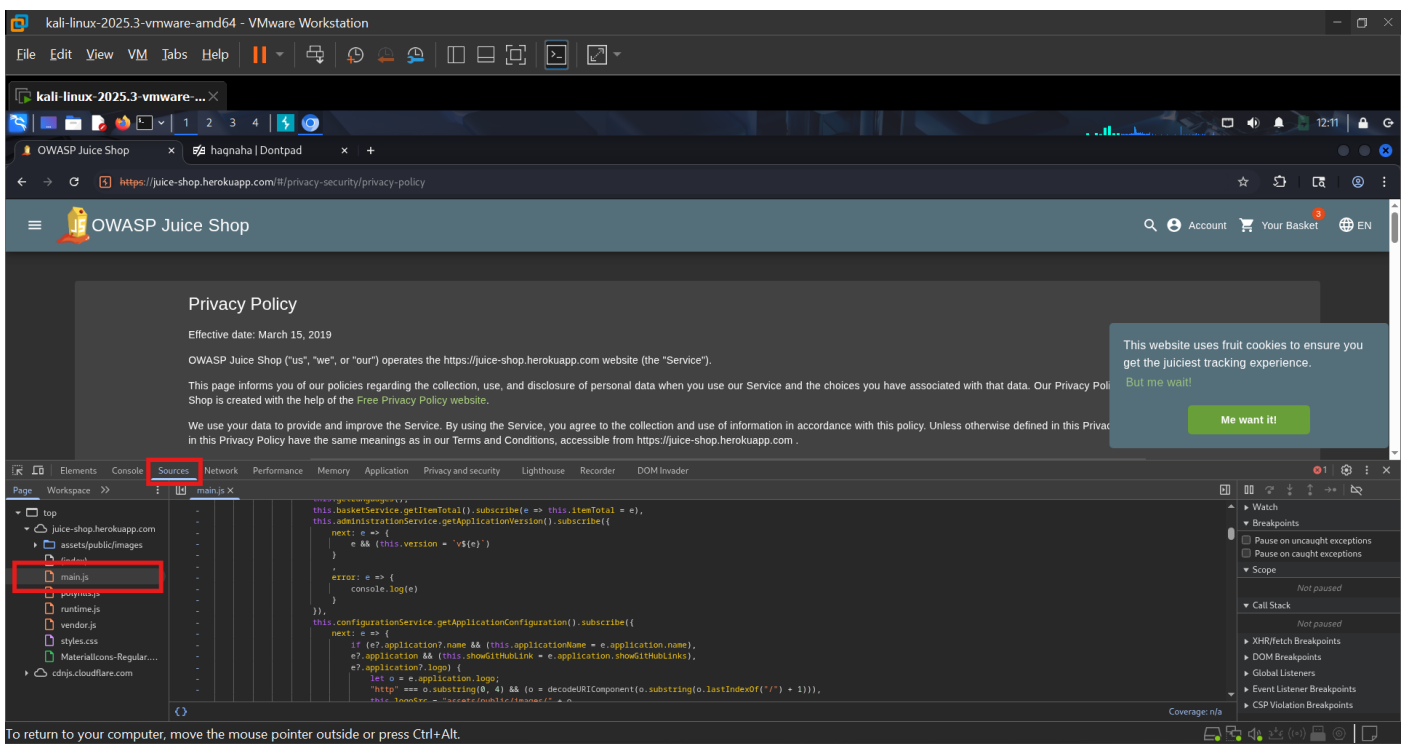
2.12 I am able to see the “Privacy Policy” now. This proves that the website has “**Privacy Policy Exposure**” vulnerability.



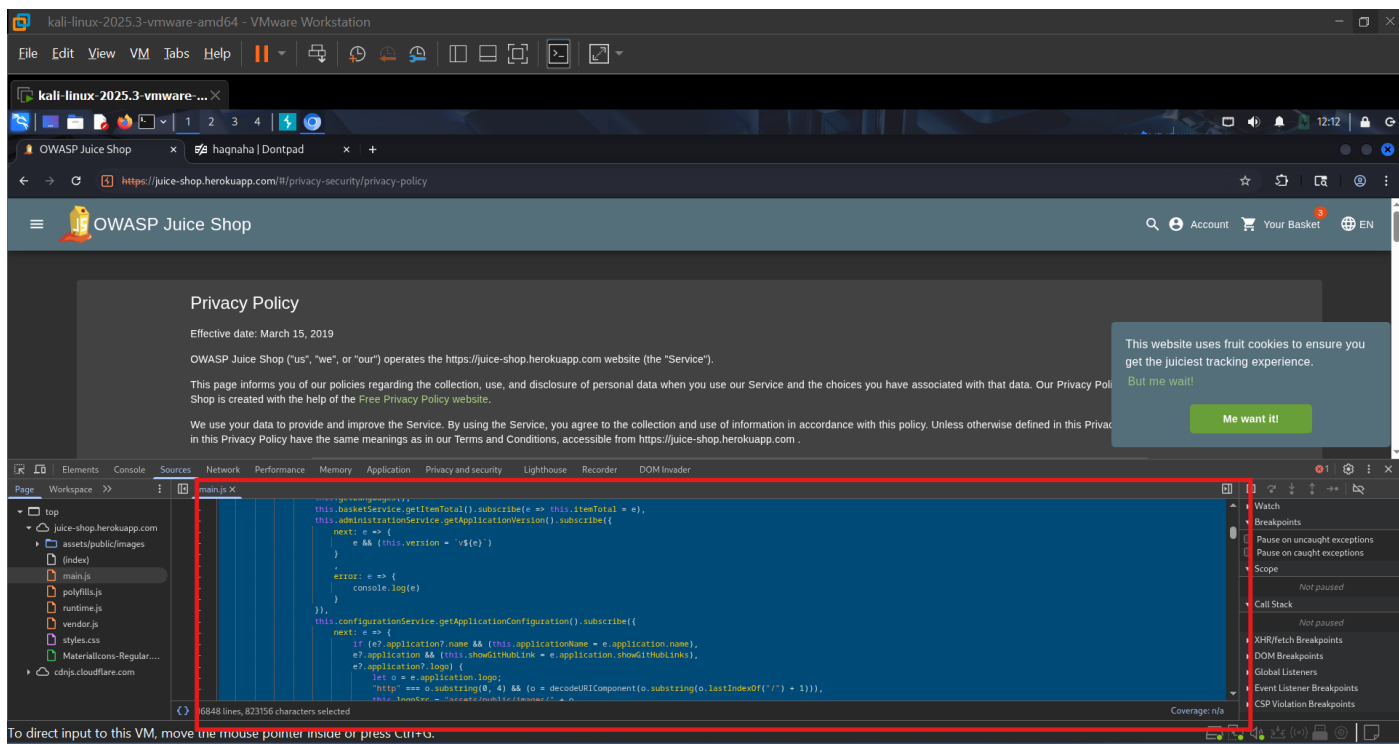
2.13 Right-clicked on the blank-space and clicked on the “**Inspect**” option.



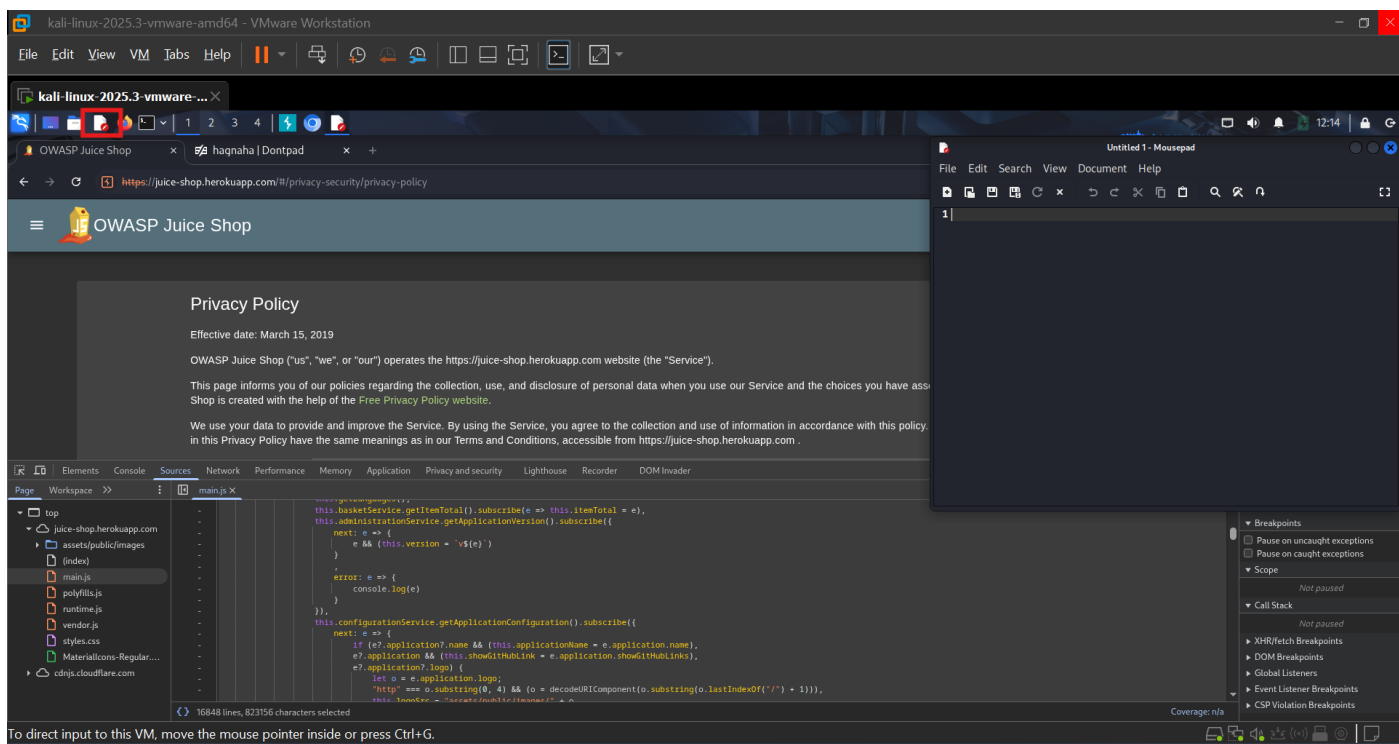
2.14 Now clicked on the “Source” section and selected the “main.js” file.



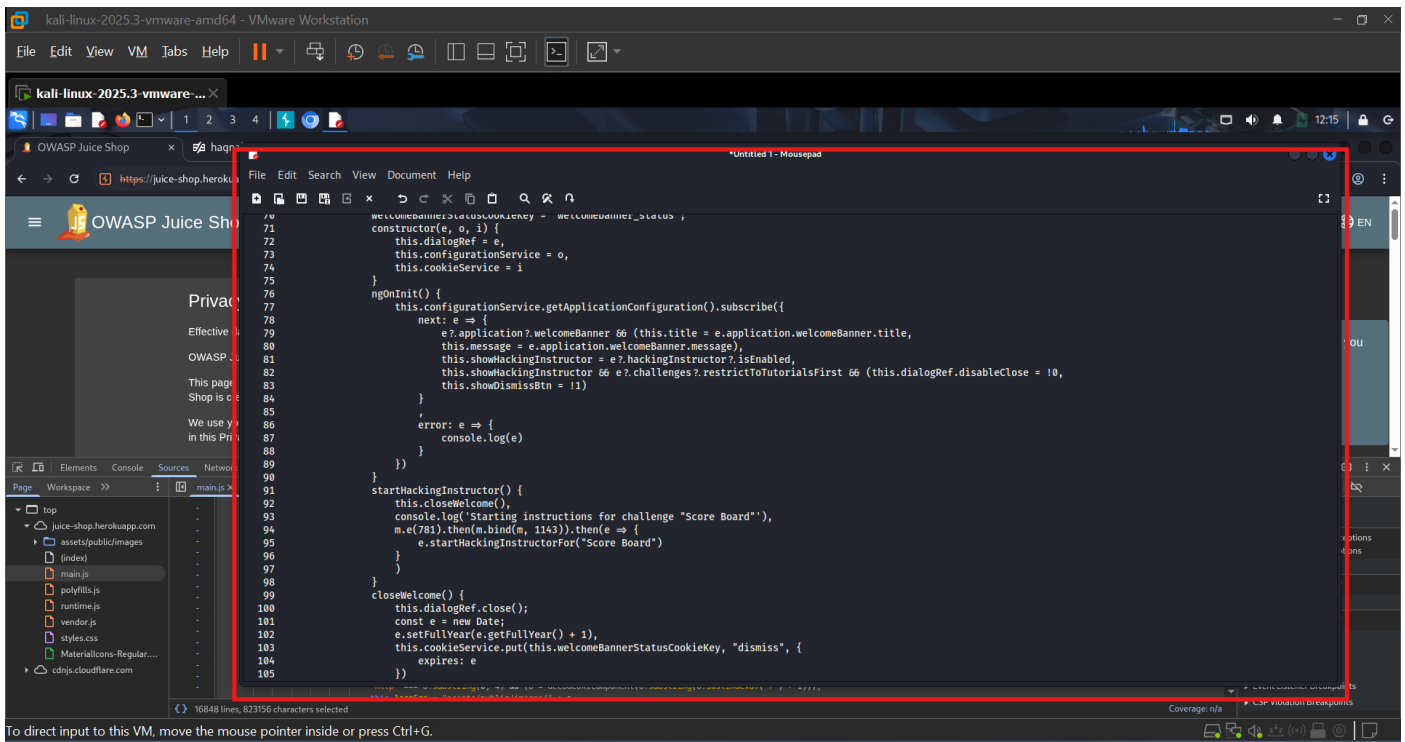
2.15 Selected the entire code (Ctrl+A) and save (Ctrl+S) it.



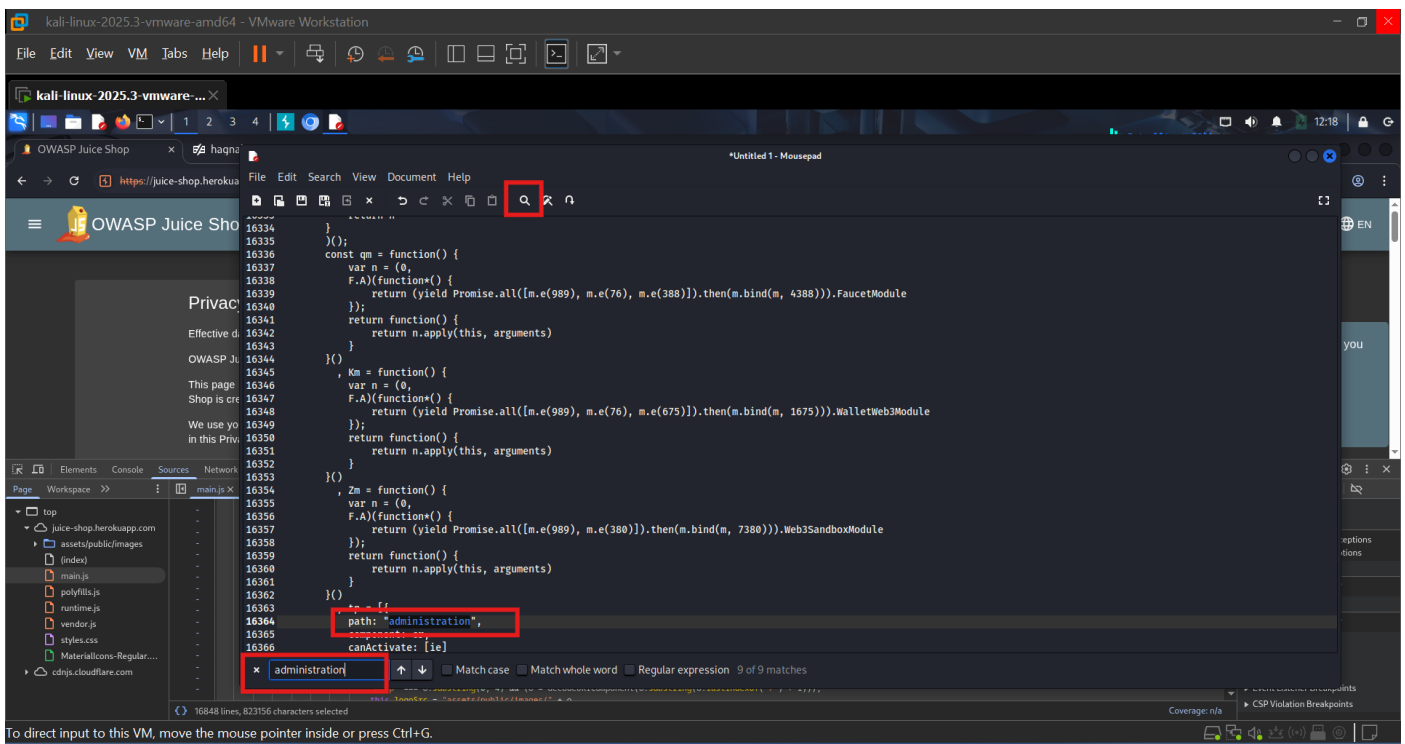
2.16 Clicked on the Editor icon.



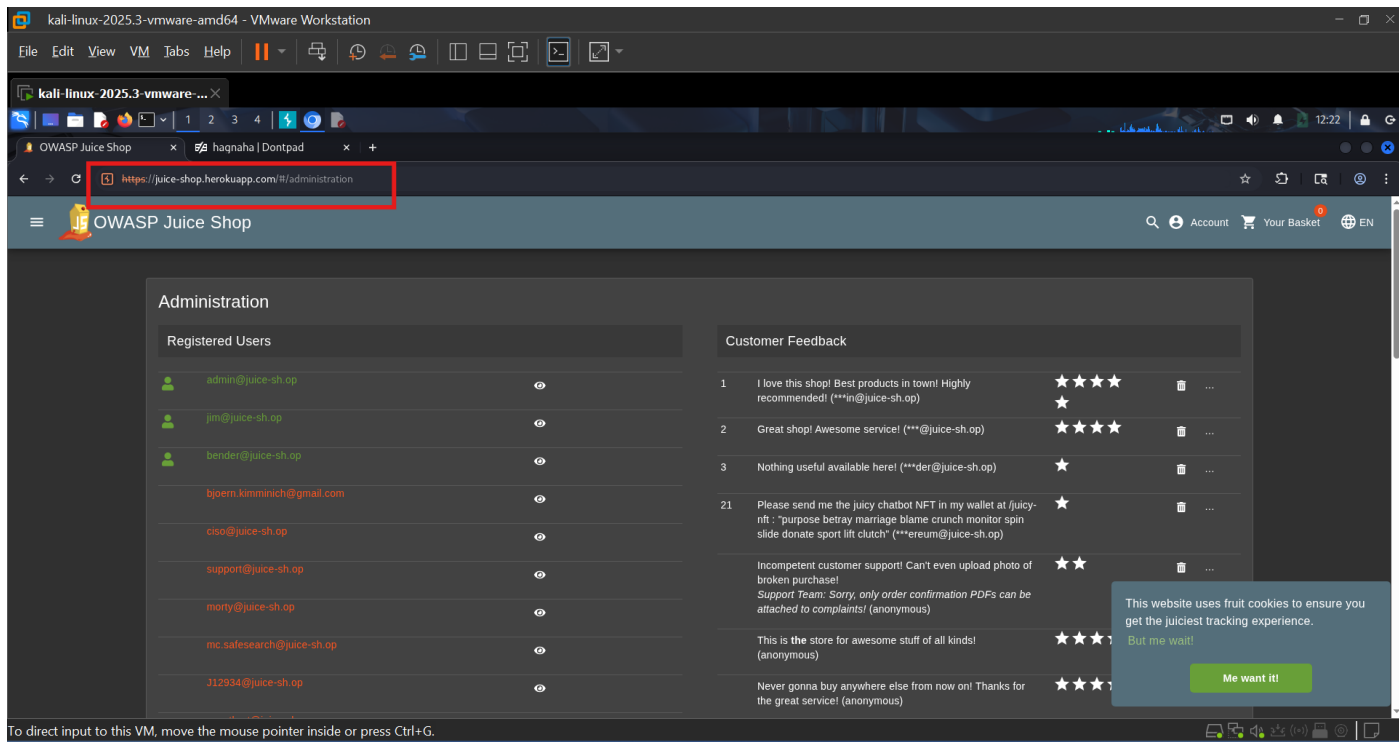
2.17 Pasted the entire script.



2.18 Clicked on the “search box” and typed “administration” in the search box, then looked for “Path: “administration””

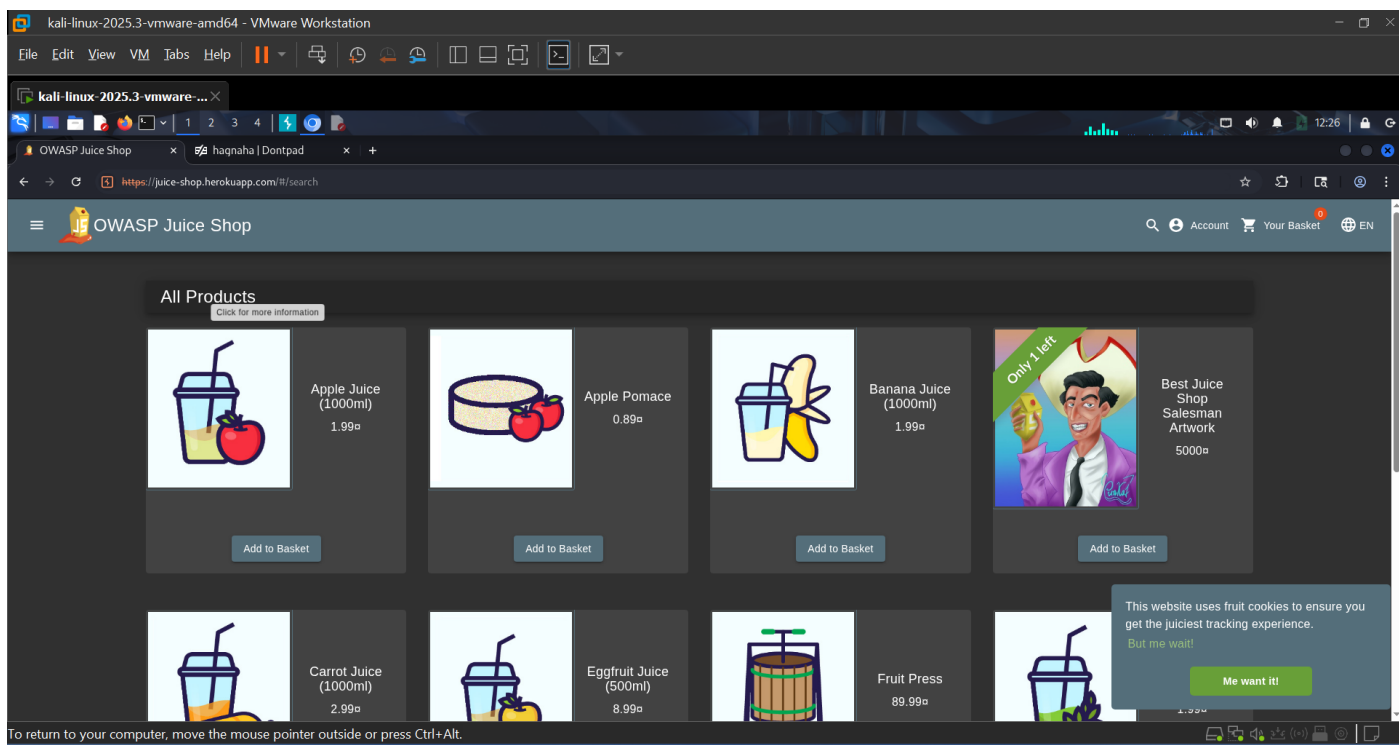


2.19 Edited the url path to “/#/administration”. I can easily access the administrator page.

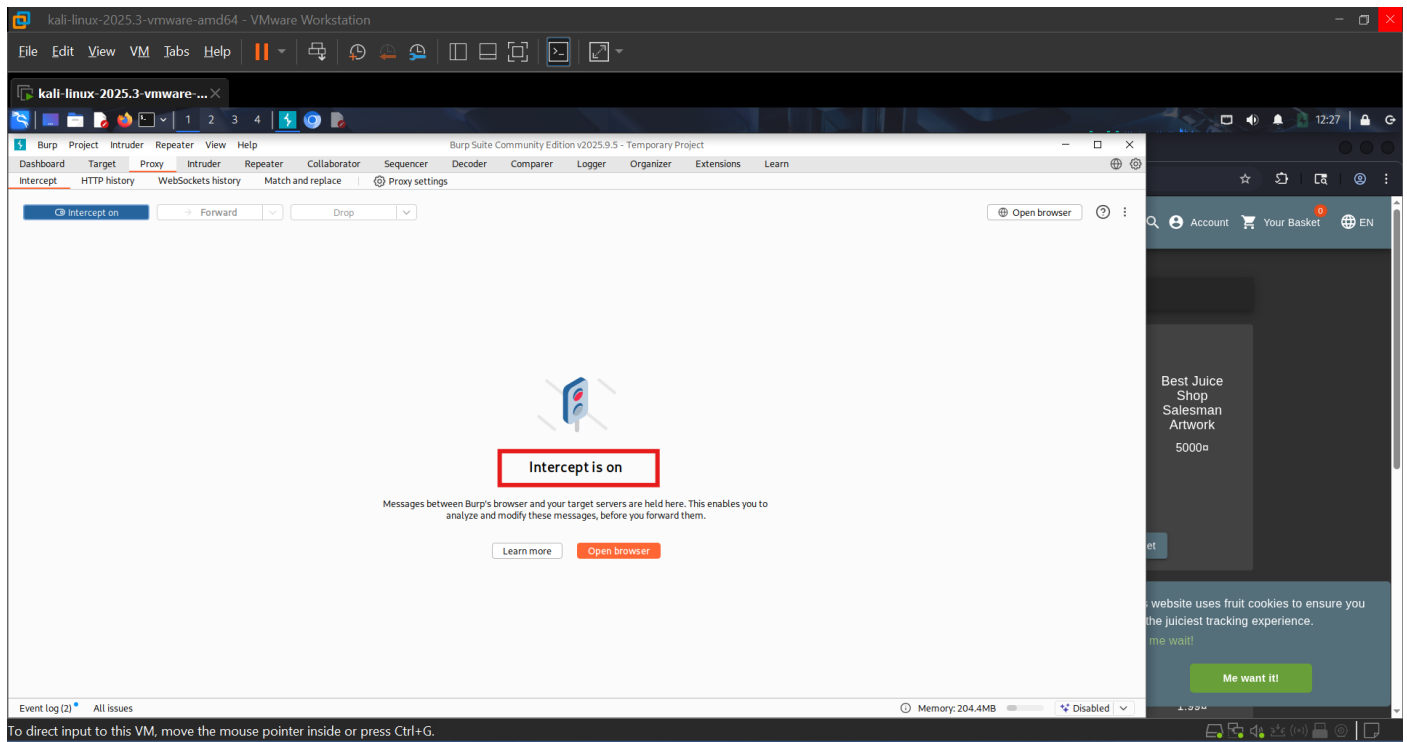


Task 3: Access Control Testing

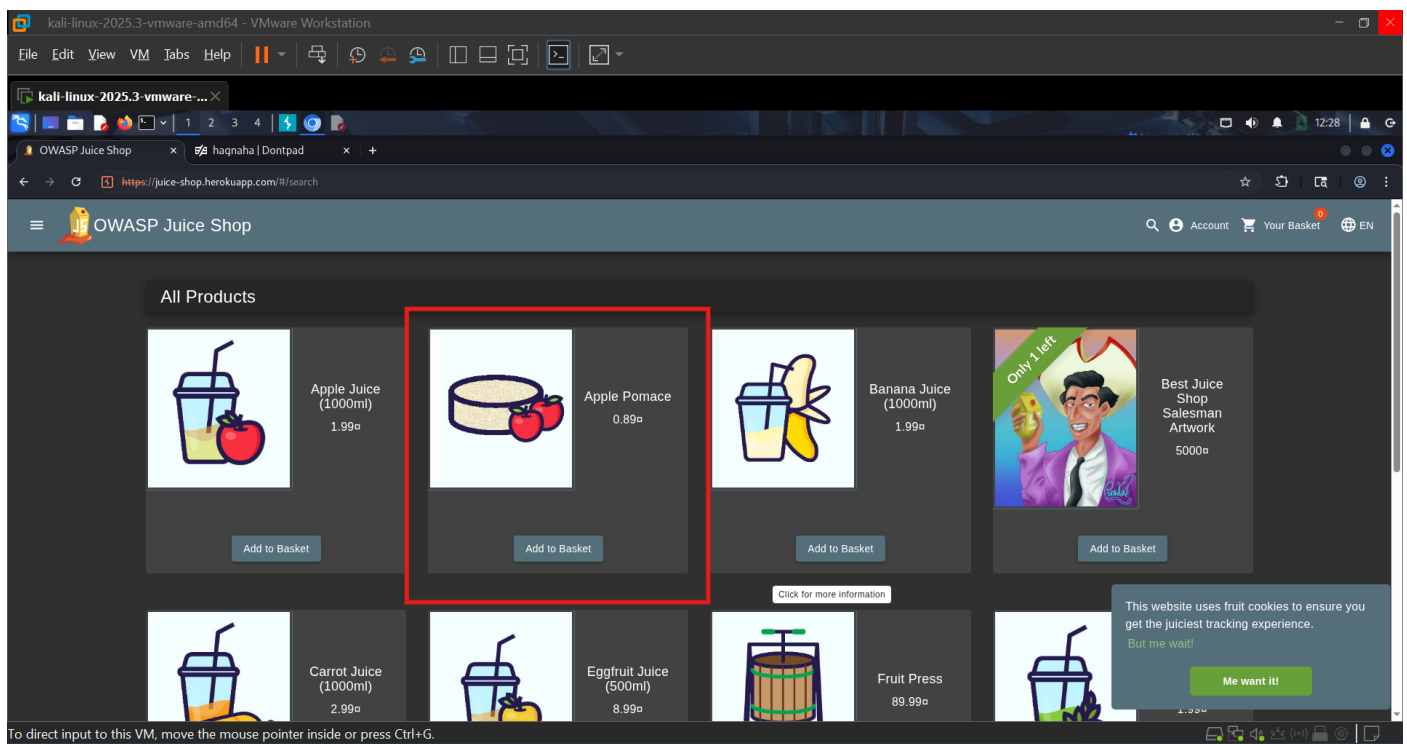
3.1 Next, returned to the home page.



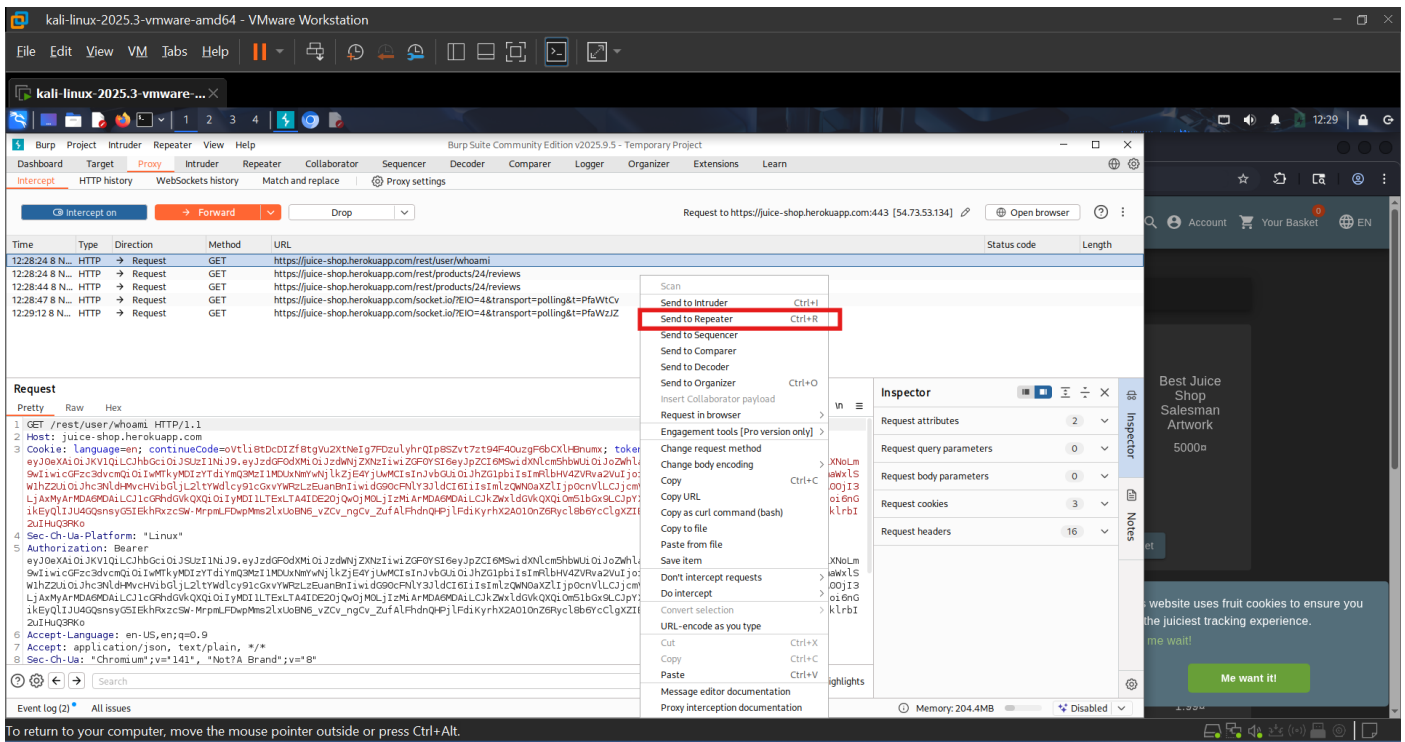
3.2 Turned on the **Interception** option.



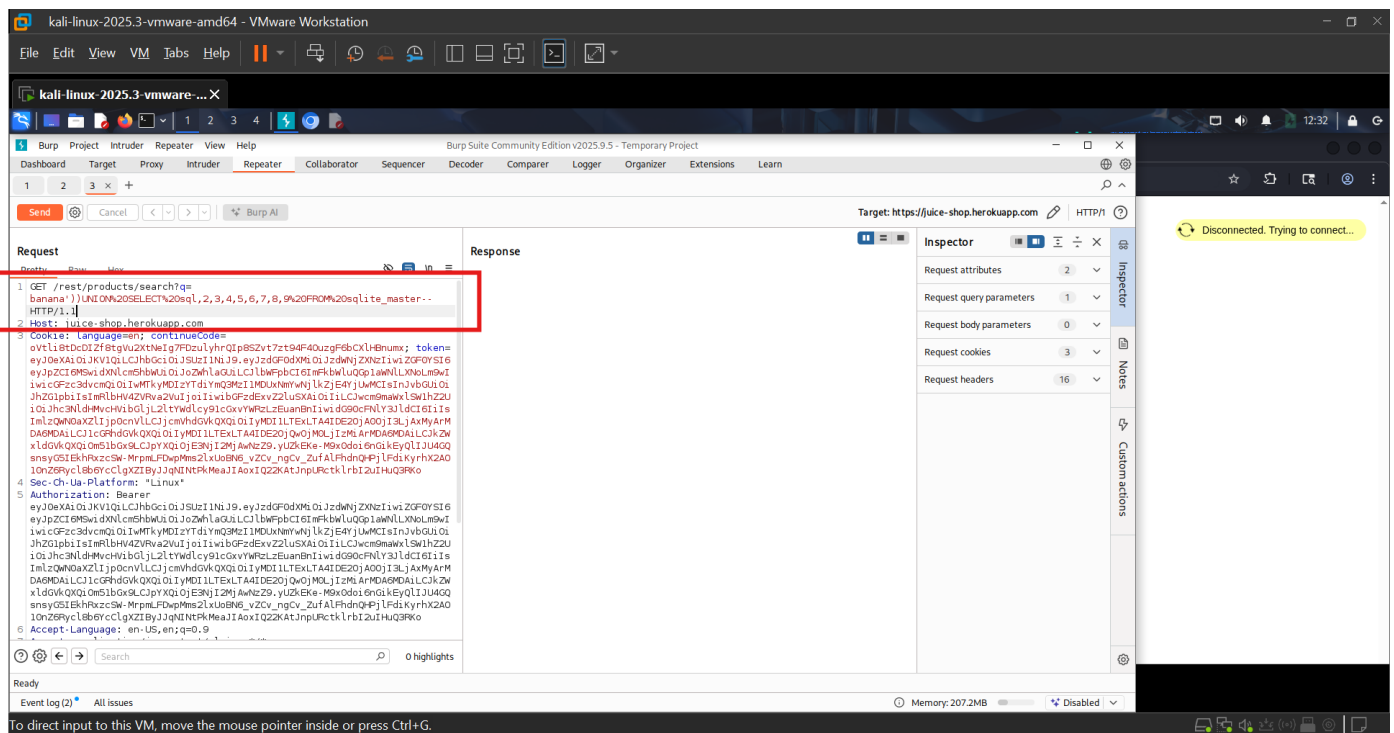
3.3 Clicked on the product in the webpage.



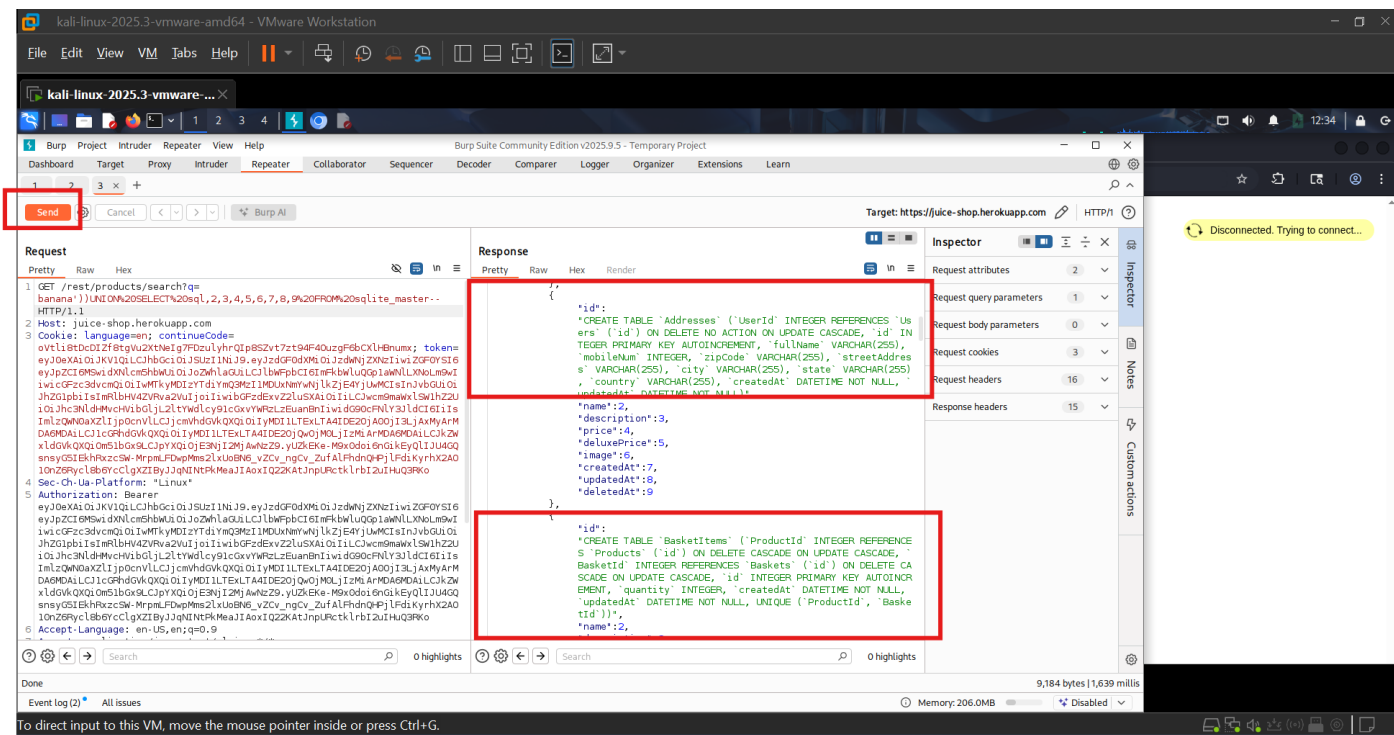
3.4 Clicked Forward until I received the heading line as “GET /rest/user/whoami HTTP/1.1”



3.6 In the request section edited the heading line and replaced it with “GET /rest/products/search?q=banana’))UNION%20SELECT%20sql,2,3,4,5,6,7,8,9%20FROM%20sqlite_master-- HTTP/1.1”

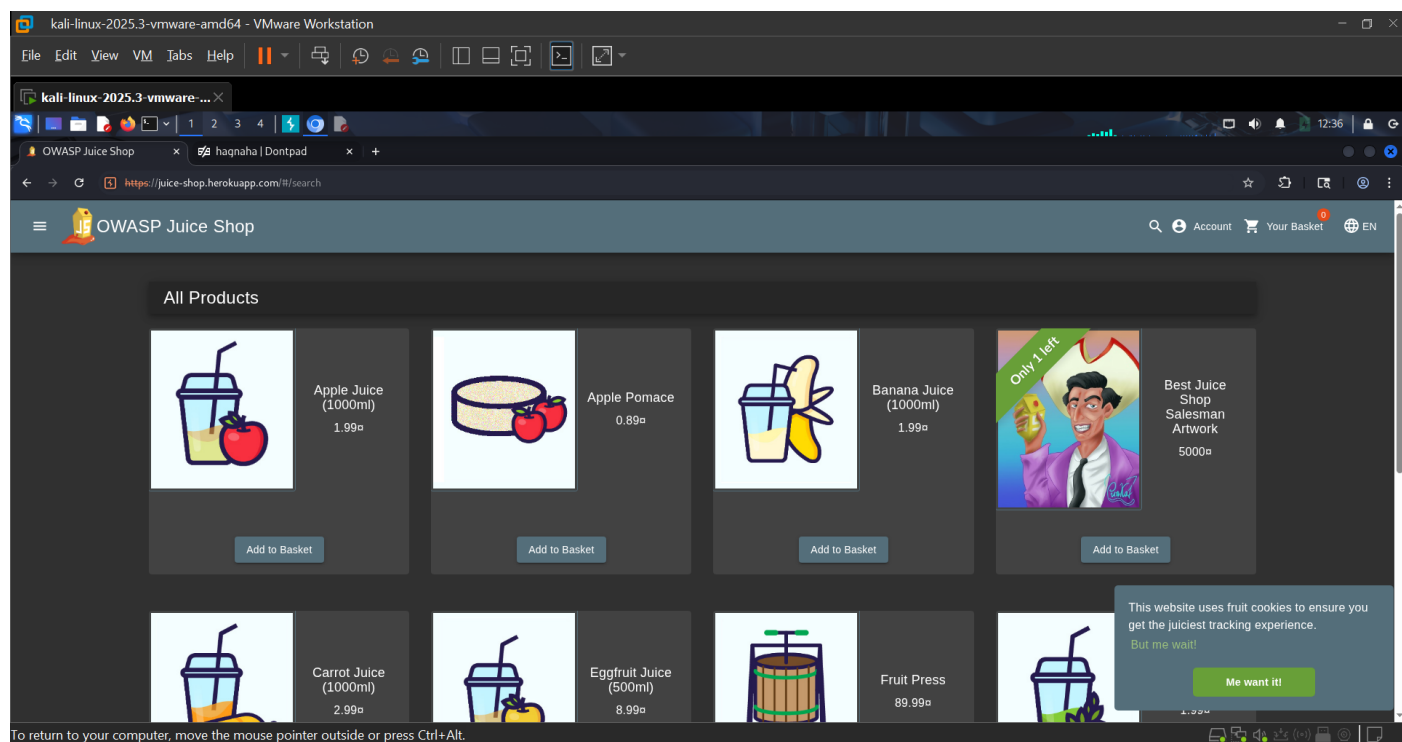


3.7 Clicked on Send

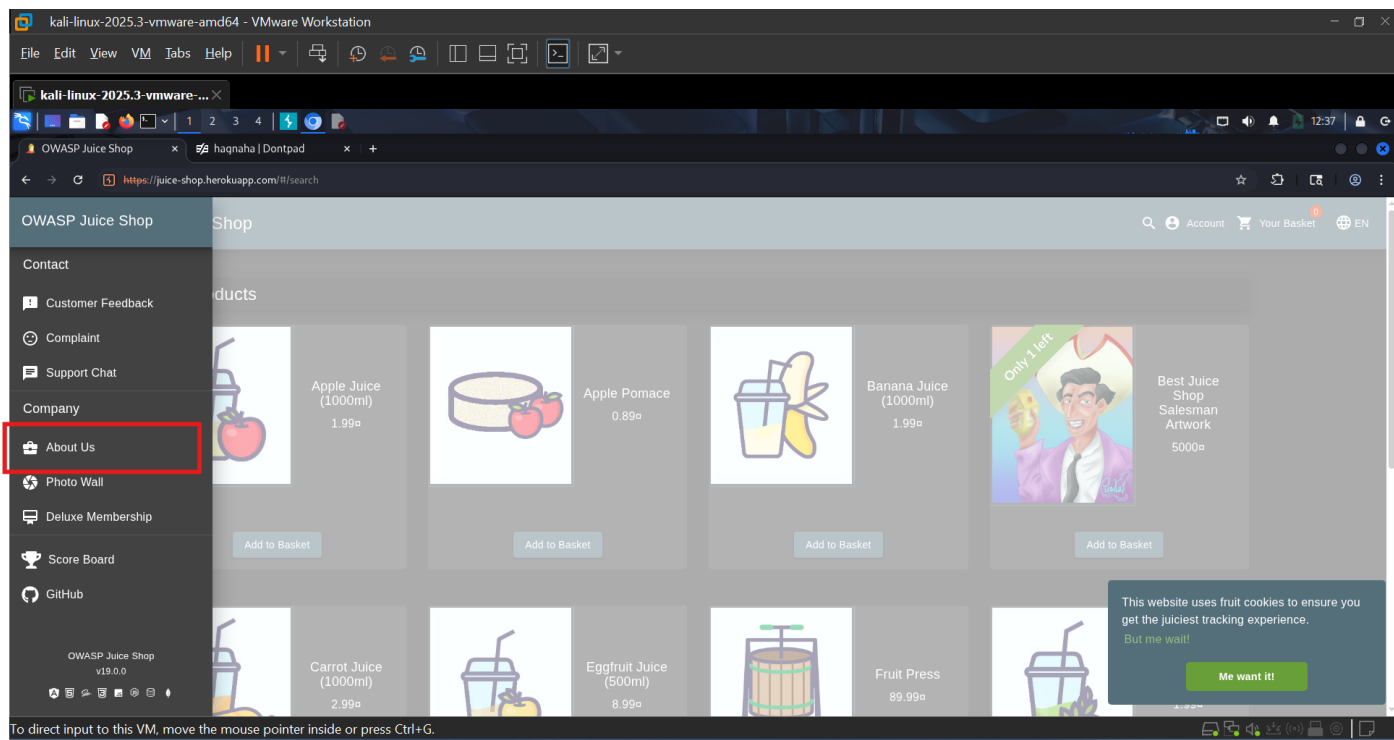


Note: I have exploited the Database Exfiltration vulnerability using SQLi.

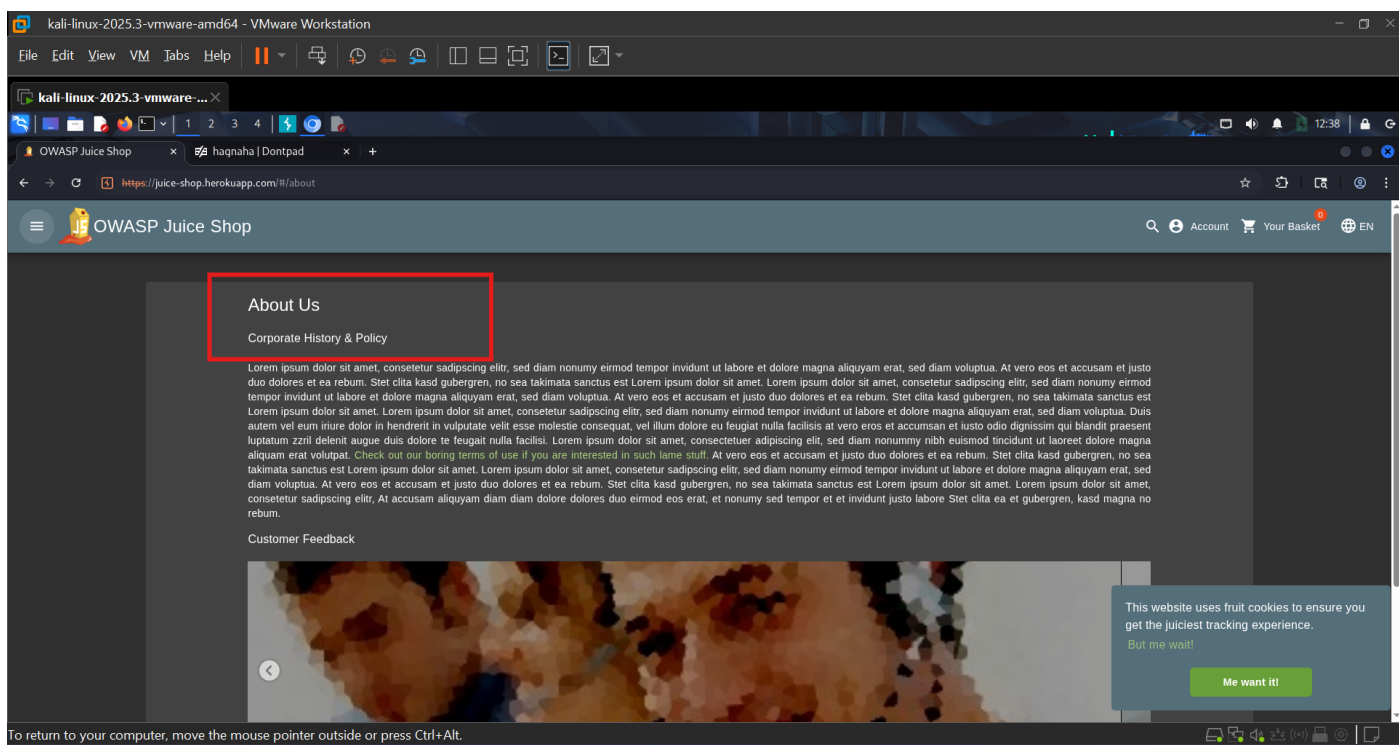
3.8 Returned back to the home page.



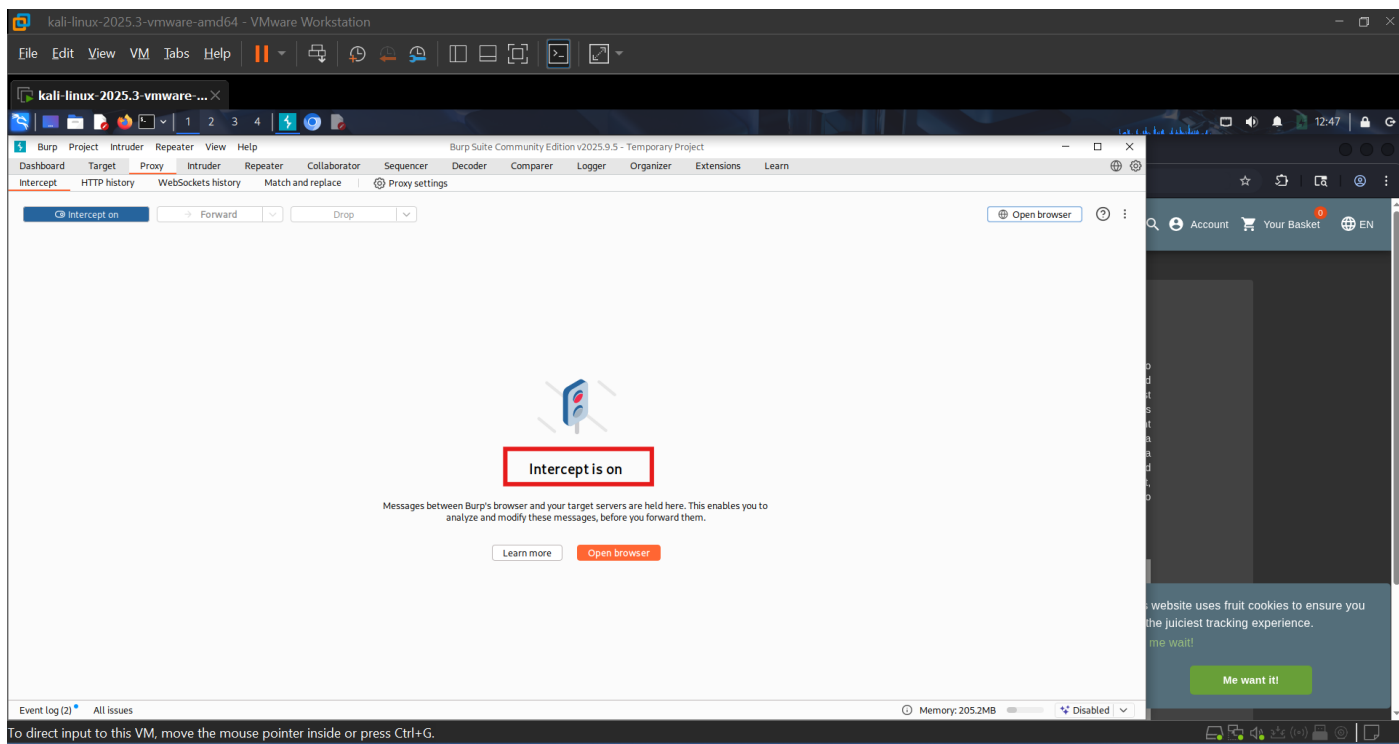
3.9 Now clicked on the parallel line icon as shown in the screenshot and then on “About Us”



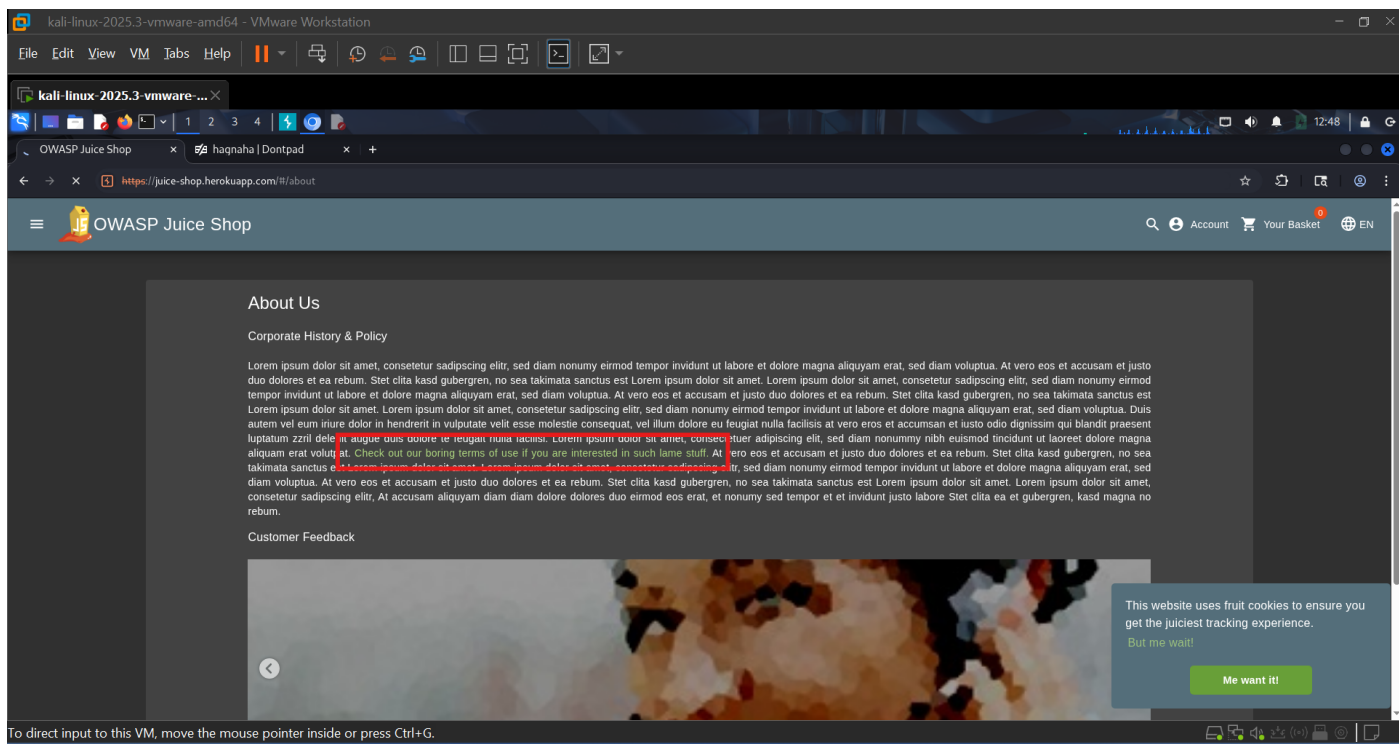
3.10 The **About Us** page appears.



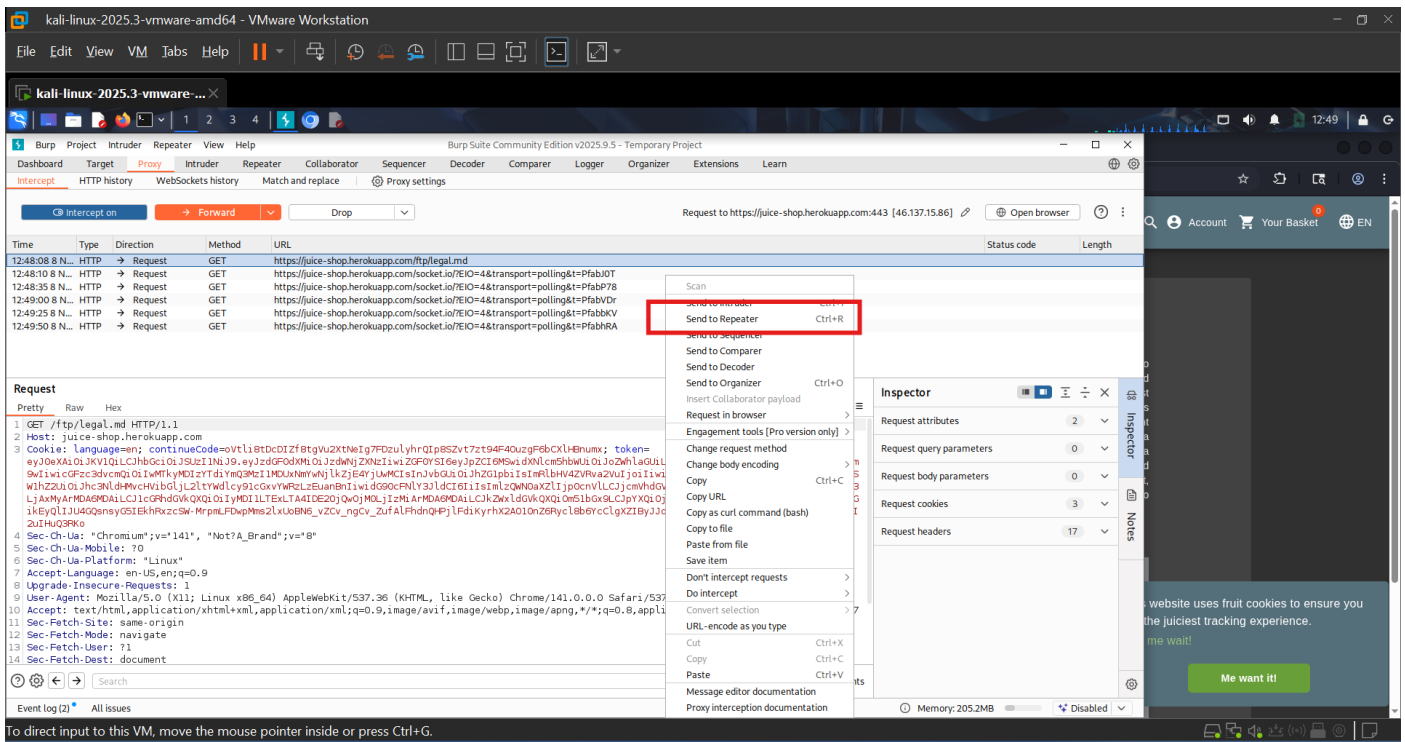
3.11 Again, turned on the **intercept** service.



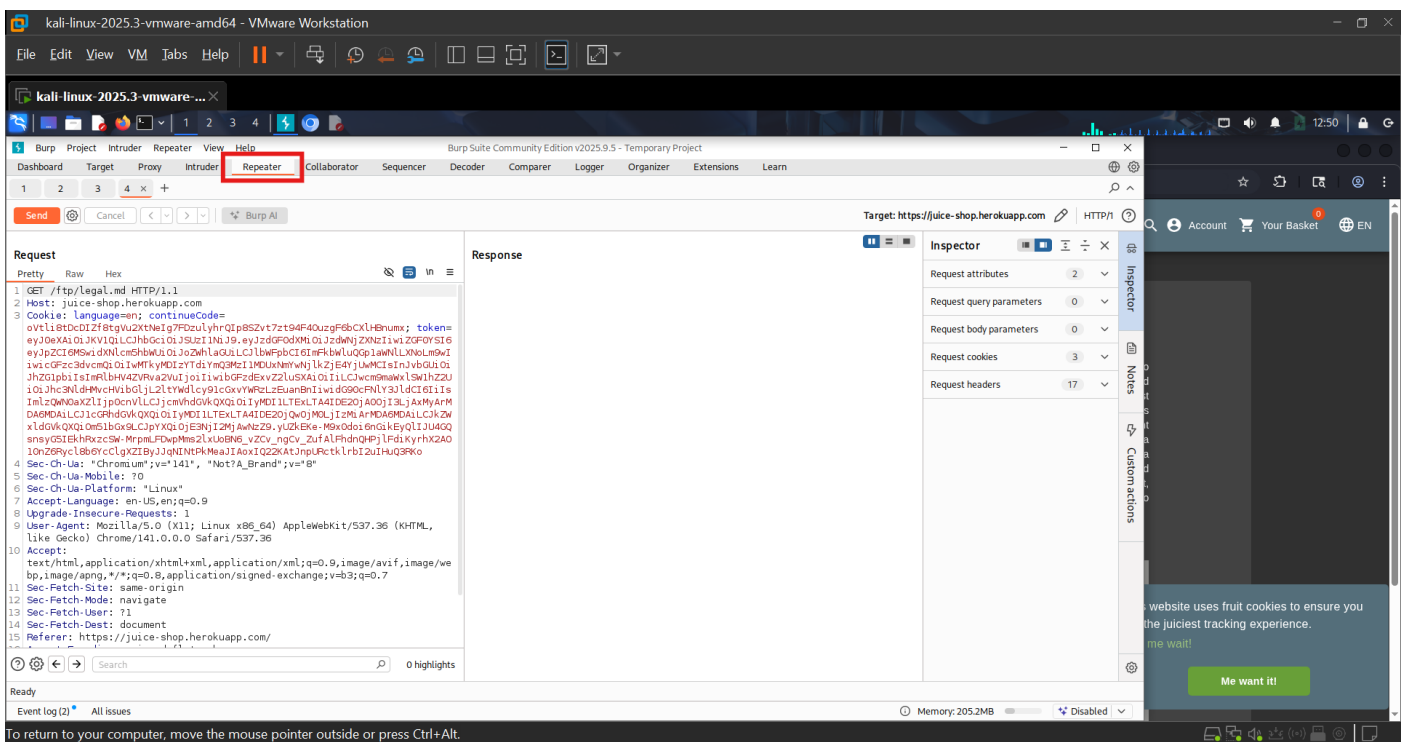
3.12 Clicked on “Check out our boring terms of use if you are interested in such lame stuff.” as highlighted in the screenshot below.



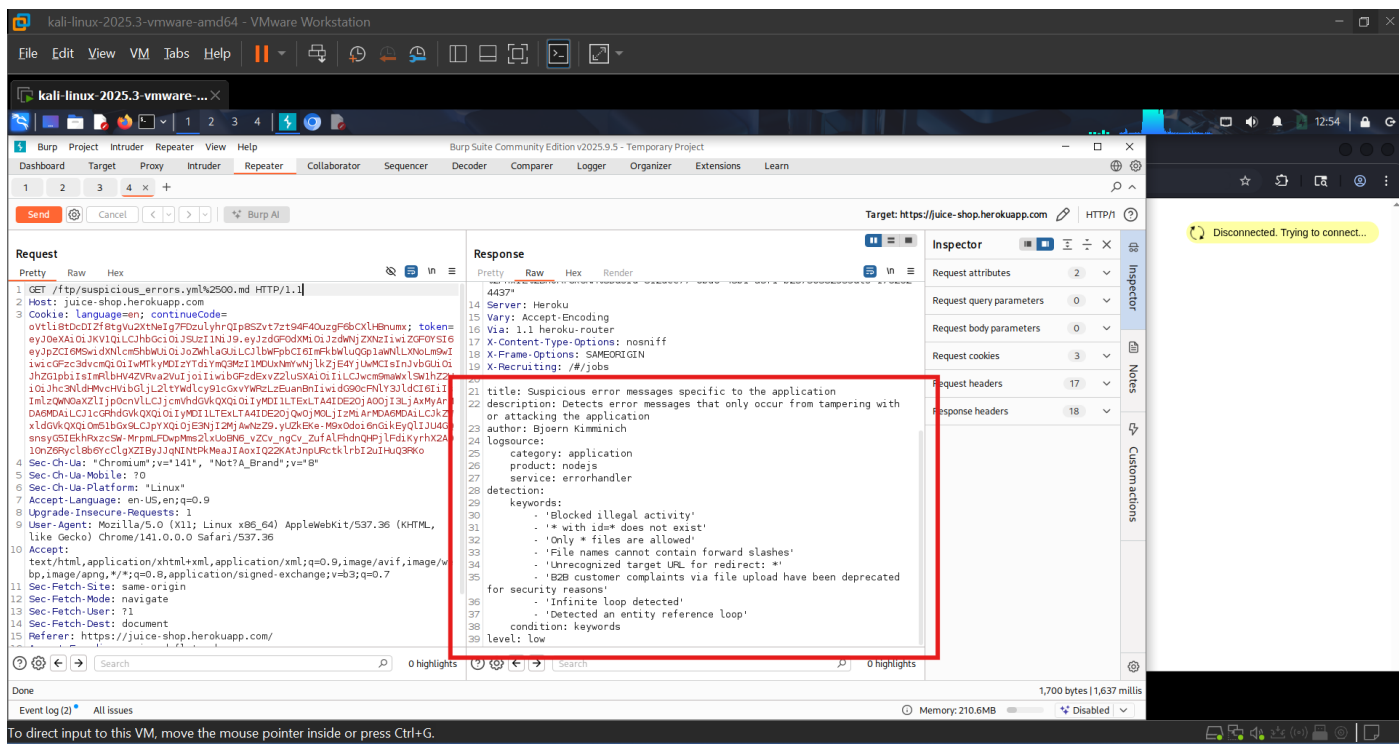
3.13 Continued clicking on forward until I get the heading line as “GET /ftp/legal.md HTTP/1.1” In the request I will see a sensitive file location: “/ftp/legal.md”.



3.15 Clicked on the Repeater.



3.16 Now removed the above directory and replaced it with “GET /ftp/suspicious_errors.yml%2500.md HTTP/1.1”. The sensitive file appear after sending the tampered request.



Note: I have exploited **Misplaced SIEM Signature File** vulnerability.

Task 4: Report

Findings and Recommendations

1. Sensitive Data Exposure

Details: Sensitive files (e.g., /ftp/legal.md and /ftp/acquisitions.md) were accessible without proper authentication.

Exploitation: An attacker could access sensitive business documents using tampered HTTP requests.

Recommendation: Implement access control policies to restrict unauthorized access to sensitive files. Use directory-level authentication or token-based access for protected resources.

2. Cross-Site Scripting (XSS)

Details: The search field was vulnerable to XSS. Executing the payload `<img src=x onerror="prompt(document.cookie)">` triggered a popup, confirming the vulnerability.

Exploitation: Malicious actors could steal cookies or inject harmful scripts.

Recommendation: Sanitize user inputs by validating and encoding special characters. Use a Content Security Policy (CSP) to mitigate the impact of XSS attacks.

3. SQL Injection (SQLi)

Details: The login page accepted SQL payloads (' or 1=1 --), allowing attackers to bypass authentication and log in as an admin.

Exploitation: Exploited to gain admin privileges and extract sensitive database information.

Recommendation: Use parameterized queries and prepared statements to prevent SQL injection. Validate and sanitize user inputs.

4. Poor Error Handling

Details: Changing API endpoints (e.g., /rest/user/whoami to /rest/tomato) resulted in internal server errors, revealing implementation details.

Exploitation: Leaked server information could aid attackers in crafting more targeted attacks.

Recommendation: Implement proper error handling mechanisms. Display generic error messages to users while logging detailed errors on the server side.

5. Privacy Policy Exposure

Details: The Privacy Policy page was accessible without authentication, even when logged in as an admin.

Exploitation: Sensitive user-related data could potentially be exposed through this vulnerability.

Recommendation: Enforce authentication checks before allowing access to sensitive resources.

6. Broken Access Control

Details: The admin panel (/#/administration) was accessible without authentication.

Exploitation: Allowed unauthorized access to administrative functions, potentially leading to system compromise.

Recommendation: Implement robust access control measures, such as role-based access control (RBAC), and enforce authentication for sensitive paths.

7. Misconfigured Database

Details: SQL injection payloads enabled database exfiltration (e.g., GET /rest/products/search?q=banana'))UNION SELECT sql,...).

Exploitation: Allowed unauthorized access to database structure and sensitive information.

Recommendation: Regularly audit database queries and configurations. Use a Web Application Firewall (WAF) to detect and block malicious SQL queries.

8. Misplaced SIEM Signature File

Details: A sensitive SIEM signature file was accessible via /ftp/suspicious_errors.yml%2500.md.

Exploitation: This file could reveal security monitoring rules, enabling attackers to bypass detection.

Recommendation: Secure sensitive files by removing them from public directories and ensuring proper access controls.

Outcome:

The project successfully identified multiple high-risk vulnerabilities such as **Sensitive Data Exposure, SQL Injection, Cross-Site Scripting, Poor Error Handling, Privacy Policy Exposure, Broken Access Control, Misconfigured Database, and Misplaced SIEM Signature Files.**

After implementing the recommended security controls:

- Sensitive resources were protected with proper authentication mechanisms.
- Input fields were sanitized to prevent injection-based attacks.
- Error responses were standardized to hide internal application details.
- Access to administrative sections and databases was restricted to authorized users only.

Conclusion:

This project provided practical exposure to real-world web security testing and vulnerability assessment. By simulating cyberattacks and analyzing weaknesses, it emphasized how even minor security flaws can lead to major data breaches. Implementing the suggested mitigations improved the website's overall security posture and aligned it with best practices in web application security.

Through this project, a deeper understanding was gained of ethical hacking methodologies, secure web development, and defensive configurations that are essential for maintaining a safe digital ecosystem. Ultimately, the project reinforced that **security is not a one-time effort but a continuous process** that requires regular monitoring, testing, and improvement.
